ReliveThePast

Final Software Engineering Report

José Pedro Andrade Machado

Universidade Lusófona do Porto

Project II

may 2025

Contents

Versions **4**

List of Figures **5**

Tables Index **6**

1 Introduction **8**
- 1.1 Purpose Of This Document . . . 8
- 1.2 Contextualization . . . 9
- 1.3 Motivation and Problem Identification . . . 10
- 1.4 System Objective . . . 11
- 1.5 Glossary . . . 12
- 1.6 Requirements format . . . 13

2 Context **33**
- 2.1 Scope . . . 33
  - 2.1.1 Relevance . . . 33
  - 2.1.2 Viability . . . 33
  - 2.1.3 Organization Structure . . . 34
  - 2.1.4 Organization Processes . . . 35
  - 2.1.5 System Support in the organization . . . 37
- 2.2 Generic Use Case Model . . . 39
- 2.3 Main exclusions . . . 41

Use Case Diagram **42**

Non-functional and functional Requirements **43**
- 4.1 Non-Functional Requirement . . . 43
- 4.2 Functional Requirement . . . 44

Technologies **45**
5.1 Technologies server-side . . . 45
5.1.1 Programming language . . . 45
5.1.2 Database Management . . . 45
5.1.3 Development Tools . . . 46
5.1.4 APIs and Integration . . . 46
5.1.5 Version Control and Backups . . . 47
5.2 Technologies client-side . . . 47
5.2.1 Programming Language and Layout Design . . . 47
5.2.2 Development Tools . . . 47
5.2.3 Accessibility and Usability Standards . . . 48

Logical and physical design **49**
6.1 Logical organization of the system . . . 49
6.1.1 Main Components . . . 50
6.1.2 External Modules . . . 51
6.1.3 Component Relationships . . . 51
6.2 Physical organization of the system . . . 52
6.2.1 Key Components . . . 52
6.2.2 Relationships and Interactions . . . 53
6.3 Database schema . . . 54
6.3.1 Entities . . . 55
6.3.2 Relationship . . . 56

Solution Developed **57**
7.1 Introduction . . . 57
7.2 Architecture . . . 58
7.2.1 Main Components . . . 59
7.2.2 Architectural Choices . . . 60
7.3 Technologies and Tools Used . . . 60
7.3.1 Tools and Platforms . . . 61
7.4 Scope of Curricular Application . . . 62
7.5 Components . . . 63
7.5.1 Firebase Realtime Database . . . 63
7.5.2 Google Maps API . . . 63
7.5.3 Google Places API . . . 63
7.5.4 Glide (GitHub) . . . 64
7.5.5 uCrop (GitHub) . . . 64
7.6 Interfaces . . . 65
7.6.1 Welcome, Login, and Sign-Up Screens . . . 65

7.6.2 Main Activity and Map Interface 67
7.6.3 Profile Screen 69
7.6.4 Create Personal POI Screen 70
7.6.5 Saved POIs Screen 72

Testing and Validation **74**
8.1 Introduction 74
8.2 Unit Tests 75
8.3 Integration Tests 76
8.4 Functional Tests 77
8.5 Usability Tests 78
8.5.1 System Usability Scale (SUS) 79
8.5.2 NPS (Net Promoter Score) 80
8.5.3 Customer Satisfaction Score (CSAT) 80

Method and Planning **81**

Results **82**
10.1 Test Results 82
10.1.1 Usability Tests 82
10.1.2 System Usability Scale (SUS) 86
10.1.3 NPS (Net Promoter Score) 89
10.1.4 Customer Satisfaction Score (CSAT) 90
10.1.5 Grading 91
10.2 Compliance with requirements 92

Conclusion **93**
11.1 Conclusion 93
11.2 Future Work 94

Bibliography **95**

Acknowledgements **96**

Versions

Version	Date	Author	Coments
1	25/11/2024	José Pedro Andrade Machado	Inicial Version
2	06/01/2025	José Pedro Andrade Machado	Software Engineering Report
3	25/04/2025	José Pedro Andrade Machado	Software Project (Interim) Report
4	19/05/2025	José Pedro Andrade Machado	Final Software Engineering Report

Table 1: Document Version History

List of Figures

1 Activity Diagram for Current Organizational Processes 36
2 Activity Diagram for System Supported Processes 38
3 Package Diagram for RelieveThePast 39
4 Use Case Diagram for ReliveThePast 42
5 Logical organization of the system 49
6 Physical organization of the system 52
7 Database Schema 54
8 Components Diagram 58
9 Welcome Screen, Sign-Up Screen , and Login Screen 65
10 Main Map View, POI Details View, Navigation Drawer 67
11 Profile Screen – User Overview and Account Management 69
12 Create POI Interface – Image Selection, Description and Map Pinning 70
13 Saved POIs Interface – List of Personalized Favorites 72
14 Gantt Diagram 81
15 Task 1-View Point Of Interest Details 83
16 Task 2-Save a Point Of Interest 83
17 Task 1-View Point Of Interest Details 84
18 Task 4 - View Your Point Of Interest 84
19 Task 5 - Edit Your Point Of Interest 85
20 Q1, Q2, Q3, Q4, Q5, Q6 87
21 Q7, Q8, Q9, Q10 88
22 NPS (Net Promoter Score) 89
23 Customer Satisfaction Score (CSAT) 90
24 Grading the app 91

Tables Index

1 Document Version History 4
2 Objectives of ReliveThePast 11
3 Glossary 12
4 Use Case: Search POIs 13
5 Use Case: View POI Details 13
6 Use Case: Save POIs 14
7 Use Case: Rate and Review POIs 14
8 Use Case: Share POIs 14
9 Use Case: Receive Notifications 15
10 Use Case: Manage Account 15
11 Use Case: View Nearby POIs 15
12 Use Case: Login/Signup 16
13 Use Case: Manage POI Data 16
14 Use Case: Verify POI Data 16
15 Use Case: Approve User Reviews 17
16 Use Case: Generate Reports 17
17 Use Case: Create Personal POI 17
18 Use Case: Edit Personal POI 18
19 States of Use Cases: UC01 to UC08 19
20 States of Use Cases: UC09 to UC15 20
21 Event Flow Descriptions (Part 1) 21
21 Event Flow Descriptions (Part 2) 22
21 Event Flow Descriptions (Part 3) 23
21 Event Flow Descriptions (Part 4) 24
22 Requirements Description for UC01 25
23 Requirements Description for UC02 25
24 Requirements Description for UC03 26
25 Requirements Description for UC04 26
26 Requirements Description for UC05 27
27 Requirements Description for UC06 27

28 Requirements Description for UC07 28
29 Requirements Description for UC08 28
30 Requirements Description for UC09 29
31 Requirements Description for UC10 29
32 Requirements Description for UC11 29
33 Requirements Description for UC12 30
34 Requirements Description for UC13 30
35 Requirements Description for UC14 31
36 Requirements Description for UC15 31
37 Requirement Priority Levels 32
38 Actors of the ReliveThePast System 39
39 Use Case Packages for the ReliveThePast System 40
40 Processes Not Supported in the ReliveThePast System 41
41 Non-Functional Requirements 43
42 Functional Requirements 44
43 Compliance with functional requirements 92

Introduction

1.1 Purpose Of This Document

ReliveThePast is a combination of the latest advancements in mobile technology and the cause of our cultural heritage preservation, achieved through an interactive User Centered experience. Produced as part of the software engineering process, the final report is the summary of the conceptual, architectural, and implementation decisions that went into creating the application. The project follows an iterative software development approach, gaining feedback, narrowing the targets focuses and expanding the technical parts allowing us to achieve a solid, robust and scalable solution.

This report extends previous versions, and gives a complete overview on the final system, including in depth analysis of its design decisions, functional requirements, technology stack, user interface aspects and testing procedures. This is the primary objective towards which the application has been designed, i.e., to improve the access to cultural and historical Points of Interest (POIs) by means of geolocated content and real time pictures. ReliveThePast, at the core, aims to enable users, mainly tourists, students and teachers to interact with local heritage in a profound way via their mobile.

With the delivery of a multimedia-rich, location-aware experience, this system closes this gap between traditional visit and the digital expectations of users today. The finished report is an example of how well-considered architecture, contemporary tools like Firebase and Kotlin, and a user-centred focus – can actually bring real-world value through software. It is both a technical demonstration and a meditation on the software engineering process by which the project took shape from idea to deployment-ready prototype.

1.2 Contextualization

With the need for an innovative technology-based solution becoming evident, the "ReliveThePast" system aims to do just that, with an emphasis placed on exploration into culture and heritage preservation. At the same time, in our digital age, geolocation-based applications have emerged as a crucial means of bridging the gap between users and their environment in the context of historical and cultural sites. With geolocation and Location-Based Services (LBS), "ReliveThePast" makes cultural experiences more traditional but more advanced with technology. The app will identify POIs in real-time as the user travels throughout virtual or physical space and serve multimedia content to make for a more engaging and educational experience while exploring.

This kind of system is in line with the emerging trends of cultural tourism and education for tourists, cultural enthusiasts of all variety, and educators at institutions wishing to engage more deeply with their heritage. "ReliveThePast" aligns with broader objectives of cultural engagement and preservation through raising awareness, appreciation, and use of local cultural and historical sites. Its combination of using modern technology along with a focus on user-centered design makes it relevant in both academic and practical spheres, as it reflects an understanding that an interactive, accessible platform is vital for engaging audiences with culture in an increasingly digital world.

1.3 Motivation and Problem Identification

With everything going digital, cultural and historical landmarks are missing out on the ease with which people can discover them. So, the mission of the "ReliveThePast" system was born. Tourists, residents, and educators alike struggle to find and engage with cultural sites, partly due to a lack of accessible, interactive roadmaps for cultural engagement. Traditional methods of exploration often do not offer that dynamic and personalized experience users are accustomed to today. In this research, we present a system that makes use of modern technologies like geolocation as well as multimedia content to enhance the accessibility, meaning, and engagement of cultural exploration based on an online platform.

The central issue this project aims to address is the underutilization and unawareness of cultural heritage sites, particularly among the younger generations who have grown up with digital-first solutions. A lot of cultural treasures are under the radar and not connected with tools we use to explore today. "ReliveThePast" solves this problem using real-time geolocation and interactive learning content to facilitate exploration in the local area. Such a feature keeps users engaged while nurturing a relationship with the local heritage, ultimately bridging the gap between conventional cultural appreciation and the requirements of a tech-savvy society.

1.4 System Objective

ReliveThePast strives to fill an ever increasing void for a global appetite for historical and cultural information in a vibrant, interactive format. Users looking for real-time location-based content are limited in option with traditional exploring of historical points of interest needing outlines or guides, limiting access to the information based on an itinerary. ReliveThePast aims to bridge this gap with an as-you-go, ever present mobile solution where users can interact with historical and cultural sites they come across which will not only improve the tourism experience but also foster appreciation and love for local culture.

It is important mobile application implemented in the organizations of tourism, education, local government in the promotion of cultural heritage, contributing to the awareness of the cultural heritage, as well as the promotion of local tourism. It is also serves as a educational aid for schools to partake in as a hands-on experience in conjunction with lessons in history and culture. By leveraging geolocation and LBS technology, data becomes a multi-faceted learning experience as each point of interest becomes a digital repository of history and multimedia files turned into an immersive, relevant curriculum that connects with people through generations.

Objective	**Brief Description**
1. **Enhance User Accessibility**	Provide historical and cultural information in a mobile format, accessible anytime, anywhere.
2. **Promote Local Tourism**	Encourage exploration of local sites by offering informative, location-based insights about nearby POIs.
3. **Educational Engagement**	Serve as a learning tool for history and cultural studies, supporting schools and institutions with practical applications.
4. **Preserve Cultural Heritage**	Digitally archive and present cultural heritage, making historical knowledge widely available.
5. **Enrich User Experience with LBS**	Utilize geolocation and location-based services to create an immersive, real-time exploration experience.

Table 2: Objectives of ReliveThePast

1.5 Glossary

Term	**Definition**
Geolocation	The identification or estimation of the real-world geographic location of a person or device using various technologies, such as GPS.
Location-Based Services (LBS)	Services that utilize the geographical location of a device to provide relevant information or functionalities, such as navigation or nearby points of interest.
Point of Interest (POI)	Specific locations that may be of interest to a user, often including landmarks, historical sites, restaurants, and other attractions.
User Interface (UI)	The means by which a user interacts with a computer program or mobile application, including the layout, buttons, and navigation elements.
API (Application Programming Interface)	A set of rules and protocols for building and interacting with software applications, allowing different systems to communicate and share data.
GIS (Geographic Information System)	A system designed to capture, store, manipulate, analyze, and display geographical data. Often used for mapping and spatial analysis.

Table 3: Glossary

1.6 Requirements format

In this document, requirements are grouped into use cases. Each use case is presented from table 4 until table 16.

Field	**Description**
Identifier	UC01 - Search POIs
Objective	Allow users to search for Points of Interest (POIs) based on location or specific criteria.
Brief Description	The user inputs a search term or criteria into the app, and the system retrieves a list of matching POIs.
Preconditions	The app must be installed, and the user must have an internet connection.
Postconditions	A list of POIs is displayed to the user.
State	Fully described

Table 4: Use Case: Search POIs

Field	**Description**
Identifier	UC02 - View POI Details
Objective	Allow users to view detailed information about a specific POI.
Brief Description	Users can access detailed descriptions, photos, reviews, and location information for a selected POI.
Preconditions	The user must select a POI from a search result or a list.
Postconditions	Detailed POI information is displayed.
State	Fully described

Table 5: Use Case: View POI Details

Field	Description
Identifier	UC03 - Save POIs
Objective	Allow users to save a POI for future reference.
Brief Description	Users can mark a POI as a favorite and save it to their profile for later access.
Preconditions	The user must be logged into their account.
Postconditions	The POI is saved under the user's account.
State	Fully described

Table 6: Use Case: Save POIs

Field	Description
Identifier	UC04 - Rate and Review POIs
Objective	Allow users to leave ratings and reviews for POIs they have visited.
Brief Description	The user submits a rating or a review, which is then associated with the POI.
Preconditions	The user must be logged into their account.
Postconditions	The review or rating is stored in the database and displayed to other users.
State	Fully described

Table 7: Use Case: Rate and Review POIs

Field	Description
Identifier	UC05 - Share POIs
Objective	Allow users to share POI details on social media or with others.
Brief Description	The user selects a sharing option, and the system provides a shareable link or interface.
Preconditions	The user must have access to the share function.
Postconditions	The POI details are shared via the chosen platform.
State	Fully described

Table 8: Use Case: Share POIs

Field	Description
Identifier	UC06 - Receive Notifications
Objective	Notify users about nearby POIs, updates, or recommendations.
Brief Description	The system sends location-based notifications or updates to the user's device.
Preconditions	Notifications must be enabled on the user's device.
Postconditions	The user receives relevant notifications.
State	Described in detail

Table 9: Use Case: Receive Notifications

Field	Description
Identifier	UC07 - Manage Account
Objective	Allow users to manage their profile and preferences.
Brief Description	Users can update personal information, change settings, and manage preferences such as notification options.
Preconditions	The user must be logged into their account.
Postconditions	Changes made by the user are saved and applied in the system.
State	Fully described

Table 10: Use Case: Manage Account

Field	Description
Identifier	UC08 - View Nearby POIs
Objective	Allow users to explore POIs near their current location.
Brief Description	The system uses geolocation to show a list of POIs close to the user's current location. The user can select a POI to view more details.
Preconditions	The user must have location services enabled on their device.
Postconditions	The user views a list of POIs near their location and can select one for more information.
State	Fully described

Table 11: Use Case: View Nearby POIs

Field	Description
Identifier	UC09 - Login/Signup
Objective	Authenticate users or allow new users to register an account.
Brief Description	The user enters credentials to log in or provides personal information to create an account. Upon successful login, the user is granted access to the system's features.
Preconditions	The user must have an internet connection, and for login, they must already have an account.
Postconditions	The user is authenticated and gains access to their account features. For new users, an account is created and logged in automatically.
State	Fully described

Table 12: Use Case: Login/Signup

Field	Description
Identifier	UC10 - Manage POI Data
Objective	Allow administrators to add, edit, or delete POI information.
Brief Description	Administrators access a backend interface to manage the POI database.
Preconditions	The administrator must be logged into the system with the appropriate permissions.
Postconditions	The POI data is updated in the system.
State	Fully described

Table 13: Use Case: Manage POI Data

Field	Description
Identifier	UC11 - Verify POI Data
Objective	Ensure the accuracy and validity of POI data.
Brief Description	Administrators review and validate POI data entries in the system.
Preconditions	The administrator must have access to the POI data.
Postconditions	The verified data is marked as accurate and updated in the database.
State	Fully described

Table 14: Use Case: Verify POI Data

Field	Description
Identifier	UC12 - Approve User Reviews
Objective	Allow administrators to approve or reject user-submitted reviews.
Brief Description	Administrators review user-submitted reviews to ensure compliance with guidelines.
Preconditions	There must be pending reviews for approval.
Postconditions	The review is either approved and published or rejected.
State	Fully described

Table 15: Use Case: Approve User Reviews

Field	Description
Identifier	UC13 - Generate Reports
Objective	Allow administrators to generate usage and data reports.
Brief Description	Administrators can create reports on POI usage, user activity, and system performance.
Preconditions	The administrator must be logged into the reporting interface.
Postconditions	A report is generated and made available for download or viewing.
State	Fully described

Table 16: Use Case: Generate Reports

Field	Description
Identifier	UC14 - Create Personal POI
Objective	Allow users to create and save their own personal Points of Interest (POIs).
Brief Description	Users can manually define a POI by entering details such as name, description, location, and optionally attaching multimedia content.
Preconditions	The user must be logged into their account and have location permissions enabled.
Postconditions	A new personal POI is saved to the user's profile and stored in the database.
State	Fully described

Table 17: Use Case: Create Personal POI

Field	**Description**
Identifier	UC15 - Edit Personal POI
Objective	Allow users to edit the details of a previously created personal POI.
Brief Description	Users can update fields such as name, description, media, or coordinates of a personal POI they previously created.
Preconditions	The user must be logged in and must be the creator of the POI.
Postconditions	The updated information is saved and replaces the previous version of the POI.
State	Fully described

Table 18: Use Case: Edit Personal POI

The state indicates how the use case is described. The possible states are those indicated in table 19.

Identifier	**State**	**Characteristic**	**Objective**
UC01	Described in Detail	Includes steps for searching POIs, defining criteria, and retrieving a list of results.	Ensure the user can efficiently search for relevant POIs.
UC02	Delineated	Describes accessing detailed information about a POI, focusing on usability for navigating data.	Provide users with comprehensive information on a POI.
UC03	Described in Detail	Describes the ability to save POIs for future reference, including data persistence.	Ensure users can easily save and retrieve favorite POIs.
UC04	Described in Detail	Adds details for users to submit reviews or ratings for POIs they've visited.	Enable users to provide feedback and interact with the community.
UC05	Delineated	Focuses on the usability of sharing POI details on social media or with others.	Allow users to share POIs seamlessly via social platforms or links.
UC06	Described in Detail	Includes steps for sending location-based notifications or updates about nearby POIs.	Notify users of POIs or updates based on their current location or preferences.
UC07	Fully Described	Covers user settings management, including updating profile data and notification preferences.	Allow users to manage their account settings effectively.
UC08	Described in Detail	Explains how the system identifies and displays nearby POIs using geolocation data.	Enable users to explore POIs close to their physical location.

Table 19: States of Use Cases: UC01 to UC08

Identifier	State	Characteristic	Objective
UC09	Fully Described	Details the authentication flow for login and signup processes, including account creation.	Authenticate users and grant them access to the app's features.
UC10	Fully Described	Covers steps for administrators to add, edit, or delete POI data in the backend.	Ensure accurate and up-to-date POI information in the system.
UC11	Described in Detail	Includes validation steps for administrators to verify and correct POI data.	Maintain the accuracy and credibility of POI data.
UC12	Outlined	Covers review moderation by administrators, including approval and rejection of user reviews.	Ensure high-quality user reviews through administrative oversight.
UC13	Fully Described	Describes steps to generate system reports on POI usage, user activity, and system performance.	Allow administrators to analyze app usage through detailed reporting.
UC14	Fully Described	Describes user interaction for manually creating a POI, including input fields, multimedia uploads, and location tagging.	Allow users to contribute custom POIs for personal use and future reference.
UC15	Fully Described	Includes the process for editing a previously created personal POI with updated content or location information.	Enable users to manage and maintain their custom POIs effectively.

Table 20: States of Use Cases: UC09 to UC15

The use case scenarios are described through event flows, which can be classified into four types:

- **Basic Flow:** The sequence of events most commonly used to achieve the objective of the use case (only one basic flow exists).
- **Alternative Flows:** Other sequences of events used to perform the use case.
- **Optional Flows:** Sequences of events that extend the basic flow of the use case.
- **Exception Flows:** Sequences of events that typically prevent the use case from achieving its objective.

The event flows are described according to the format of Table 21.

Use Case	**Description**	**Extension Points**
UC01		
Step 1: Input Search Term	User inputs a term into the search bar to find POIs.	If the term is invalid, prompt the user to enter a valid term.
Step 2: Process Query	System processes the search term and queries the database.	
Step 3: Display Results	System displays a list of POIs matching the search term.	If no results are found, suggest nearby POIs.
UC02		
Step 1: Select POI	User selects a POI from the search results.	
Step 2: Display Details	System displays detailed information about the selected POI.	
UC03		
Step 1: Select POI	User selects a POI from the search results.	
Step 2: Save to Profile	System saves the selected POI to the user's profile.	If the user is not logged in, the system prompts for login.

Table 21: Event Flow Descriptions (Part 1)

Use Case	Description	Extension Points
UC04		
Step 1: Select POI	User selects a POI to rate or review.	If the user is not logged in, the system prompts for login.
Step 2: Submit Feedback	User submits a rating or review, which the system stores.	
UC05		
Step 1: Select Sharing Option	User chooses to share POI details via social media or messaging.	
Step 2: Generate Link	System generates a shareable link or interface.	
UC06		
Step 1: Detect Nearby POIs	System identifies POIs close to the user's location.	If geolocation is disabled, notify the user to enable it.
Step 2: Send Notification	System sends a notification to the user about the POIs.	
UC07		
Step 1: Access Settings	User navigates to account settings.	
Step 2: Modify Preferences	User updates preferences (e.g., notification options) or account details.	
UC08		
Step 1: Detect Location	System uses geolocation to determine the user's current location.	If geolocation is disabled, notify the user to enable it.
Step 2: Display Nearby POIs	System displays a list of POIs near the user's location.	

Table 21: Event Flow Descriptions (Part 2)

Use Case	Description	Extension Points
UC09		
Step 1: Enter Credentials	User inputs login credentials or registers for an account.	If credentials are invalid, prompt the user to re-enter them.
Step 2: Authenticate User	System authenticates the user and grants access to their account.	If authentication fails repeatedly, suggest a password reset.
UC10		
Step 1: Select POI	Administrator selects a POI from the database.	
Step 2: Modify POI Details	Administrator updates the POI details (e.g., description, location).	
Step 3: Save Changes	System saves the updated POI information.	
UC11		
Step 1: Review POI Data	Administrator reviews POI data flagged for verification.	
Step 2: Approve or Reject	Administrator approves verified data or rejects invalid data.	
UC12		
Step 1: Access Pending Reviews	Administrator accesses the list of pending reviews.	If a review violates guidelines, mark it for deletion.
Step 2: Approve or Reject Reviews	Administrator approves or rejects user-submitted reviews.	
UC13		
Step 1: Select Report Type	Administrator selects the type of report (e.g., usage, activity).	
Step 2: Generate Report	System compiles the report and makes it available for download.	

Table 21: Event Flow Descriptions (Part 3)

Use Case	Description	Extension Points
UC14		
Step 1: Open Create POI Interface	User navigates to the screen for creating a personal POI.	If the user is not logged in, prompt for authentication.
Step 2: Enter POI Details	User inputs the POI's name, description, and location; optionally adds images.	If required fields are left blank, prompt the user to complete them.
Step 3: Save POI	The system saves the POI to the user's personal database.	Display confirmation message.
UC15		
Step 1: Select Personal POI	User selects an existing personal POI from their saved list.	If the POI does not belong to the user, restrict access.
Step 2: Edit POI Details	User modifies details like name, description, images, or location.	If location is not valid, notify the user to adjust coordinates.
Step 3: Save Changes	System updates the existing POI with new information.	Show success message after update.

Table 21: Event Flow Descriptions (Part 4)

After indicating the event flows, the list of use case requirements follows. Each requirement is presented in the format of Tables 22 to 34.

Identifier	**UC01**
Priority	Essential
Summary	The system must allow users to search for POIs using a keyword or location.
Description	Users should input a search term or use their geolocation to find relevant POIs stored in the system database.
Motivation	To enable users to quickly locate historical and cultural points of interest.
Additional Information	Integration with search algorithms and geolocation APIs is required. Results should be retrieved efficiently.
Implementation Suggestions	Use a full-text search engine or database index for POIs, and integrate geolocation services.

Table 22: Requirements Description for UC01

Identifier	**UC02**
Priority	Essential
Summary	The system must display detailed information about a selected POI.
Description	When a user selects a POI, the system should provide comprehensive details such as description, images, and location.
Motivation	To allow users to make informed decisions about visiting a POI based on its significance.
Additional Information	Include support for multimedia content (e.g., images, videos) and map integration.
Implementation Suggestions	Use a structured database to store details and multimedia files for each POI.

Table 23: Requirements Description for UC02

Identifier	**UC03**
Priority	Conditional
Summary	The system should allow users to save POIs for future reference.
Description	Users should be able to save POIs to a personal list or favorites within their profile.
Motivation	To enhance user experience by enabling them to bookmark places of interest.
Additional Information	Saved POIs should be persistent across sessions for logged-in users.
Implementation Suggestions	Store saved POIs in a user-specific database table. Include an interface for managing saved items.

Table 24: Requirements Description for UC03

Identifier	**UC04**
Priority	Conditional
Summary	The system should allow users to rate and review POIs.
Description	Users should be able to submit ratings (e.g., stars) and text reviews for POIs they've visited.
Motivation	To build a community-driven feedback system for improving content relevance and quality.
Additional Information	Reviews must be moderated to filter out offensive or spam content.
Implementation Suggestions	Include a review database table with user IDs, POI IDs, and timestamps for each review.

Table 25: Requirements Description for UC04

Identifier	**UC05**
Priority	Optional
Summary	The system should allow users to share POIs on social media.
Description	Users should be able to generate shareable links or post directly to platforms such as Facebook or Twitter.
Motivation	To enable users to recommend POIs to their friends and family.
Additional Information	Ensure compliance with social media platform APIs and sharing guidelines.
Implementation Suggestions	Use dynamic URL generation for POI pages and integrate platform-specific sharing buttons.

Table 26: Requirements Description for UC05

Identifier	**UC06**
Priority	Conditional
Summary	The system should send notifications about nearby POIs to users.
Description	Users should receive push notifications about points of interest in their vicinity.
Motivation	To enhance the user experience by providing timely, location-based recommendations.
Additional Information	Notifications should respect user preferences and opt-in settings.
Implementation Suggestions	Use geofencing and notification APIs to trigger alerts when users enter specified areas.

Table 27: Requirements Description for UC06

Identifier	**UC07**
Priority	Essential
Summary	The system must allow users to manage their account settings.
Description	Users should be able to update their profile, preferences, and notification settings.
Motivation	To provide users with control over their personal information and preferences.
Additional Information	Ensure compliance with data privacy regulations such as GDPR.
Implementation Suggestions	Implement a user management module with secure access controls.

Table 28: Requirements Description for UC07

Identifier	**UC08**
Priority	Essential
Summary	The system must display POIs close to the user's current location.
Description	Based on geolocation, the system should dynamically generate a list of nearby POIs.
Motivation	To offer users relevant recommendations based on their proximity to historical and cultural sites.
Additional Information	Use geolocation APIs to pinpoint the user's location accurately.
Implementation Suggestions	Query the database using spatial queries and calculate distances between user and POIs.

Table 29: Requirements Description for UC08

Identifier	**UC09**
Priority	Essential
Summary	The system must allow users to create an account or log in to access personalized features.
Description	Users should be able to register, log in, or reset their password securely.
Motivation	To manage user-specific data and preferences securely.
Additional Information	Ensure compliance with secure password storage and authentication standards.
Implementation Suggestions	Use hashed passwords and token-based authentication.

Table 30: Requirements Description for UC09

Identifier	**UC10**
Priority	Essential
Summary	Administrators must be able to manage and update POI information.
Description	Admins should add, edit, or delete POI data to ensure accuracy.
Motivation	To maintain a high-quality and up-to-date POI database.
Additional Information	Provide an admin dashboard with CRUD (Create, Read, Update, Delete) capabilities.
Implementation Suggestions	Implement role-based access control for administrator privileges.

Table 31: Requirements Description for UC10

Identifier	**UC11**
Priority	Conditional
Summary	Administrators must verify and approve POI data.
Description	Admins should review flagged or newly added POI entries for accuracy.
Motivation	To ensure the database contains valid and accurate information.
Additional Information	Include moderation tools for admins to easily review POI data.
Implementation Suggestions	Allow admins to flag and track POIs requiring further review.

Table 32: Requirements Description for UC11

Identifier	**UC12**
Priority	Conditional
Summary	Administrators must approve or reject user-submitted reviews.
Description	Admins should moderate user reviews to filter out inappropriate content.
Motivation	To maintain a respectful and helpful review system for all users.
Additional Information	Provide a queue for pending reviews for admin processing.
Implementation Suggestions	Integrate a spam filter and AI-based content moderation tools.

Table 33: Requirements Description for UC12

Identifier	**UC13**
Priority	Optional
Summary	The system should allow administrators to generate usage and activity reports.
Description	Admins should select report types and view data visualizations for POI usage, user activity, etc.
Motivation	To provide actionable insights for improving system performance and user engagement.
Additional Information	Allow export of reports in common formats (e.g., PDF, Excel).
Implementation Suggestions	Integrate a reporting module with charts and filters for dynamic report generation.

Table 34: Requirements Description for UC13

Identifier	**UC14**
Priority	Conditional
Summary	The system should allow users to create their own personal Points of Interest (POIs).
Description	Users can input custom POI information including name, location, description, and multimedia attachments.
Motivation	To allow users to personalize their experience and record places of personal significance.
Additional Information	Personal POIs are private and only visible to the creating user unless sharing is added later.
Implementation Suggestions	Use a dedicated table in the database linked to the user ID for storing custom POIs. Allow multimedia upload through existing Firebase integrations.

Table 35: Requirements Description for UC14

Identifier	**UC15**
Priority	Conditional
Summary	The system should allow users to edit the details of a previously created personal POI.
Description	Users can modify personal POIs by changing the name, description, coordinates, or attached media.
Motivation	To give users flexibility to keep their saved locations up to date or correct errors.
Additional Information	Only the user who created the POI should be authorized to make edits.
Implementation Suggestions	Provide an edit screen within the user's saved POIs list; validate changes and overwrite the original POI record upon saving.

Table 36: Requirements Description for UC15

The indication of all elements is mandatory except for additional information and suggestions. The priority of requirements is as indicated in table 37.

Grade	**Meaning**
Essential	The product cannot be accepted until the requirement is implemented as described.
Conditional	The requirement improves the product, but it does not make the product unacceptable if not implemented.
Optional	Represents a feature that could be interesting, giving the manufacturer the opportunity to propose functionalities not included in the initial specification.

Table 37: Requirement Priority Levels

Context

2.1 Scope

2.1.1 Relevance

Using innovative technology to present historical places, time travel, sound, and other sensations, the "ReliveThePast" system approaches this issue and demonstrates the relevance of its implementation. Indeed, with the integration of geolocation, multimedia content, and an easy/friendly interface, the system allows cultural exploration to be more accessible and engaging, thus successfully addressing the underutilization of cultural treasures. Through surveys and expert feedback, the project's feasibility and significance are validated. Participants highlight their development as a factor leading to an extended awareness and appreciation of cultural heritage. They represent its ability to connect two worlds; how it balances traditional cultural appreciation and an audience accustomed and even spoiled by a digitally dynamic world that demands immediacy in experience, and thus further its pragmatism and longevity.

2.1.2 Viability

The potential of "ReliveThePast" is based on the successful implementations, sustainability, and scale-up of the project beyond an academic project. Following econometric criteria, the solution shows a potential to last, to stay hot, and to be relevant in the long term. Leveraging affordable technologies like Firebase for backend and Kotlin for development, the system is financially viable by allowing it to reach low setup and ongoing costs. Their architecture is designed to be highly scalable to enable them to expand, both in the number of users, features, and geographical footprints. Such a flexible and practical project plan, preventing the academic origin of the project from shaping it as a closed loop, ensures that the original vision can become a stand-alone platform in itself and evolve into something to continuously improve .

2.1.3 Organization Structure

The organization responsible for acquiring and using ReliveThePast is designed in a way that encourages smooth integration of technology with cultural and historical discovery. It is comprised of three primary groups; end-users, a managing team, and a technical infrasture team. Target customers — tourists, history enthusiasts, educational institutions, and cultural organizations who will use the app to access, save, and share historical POIs These users might use the front-end functionalities in the system, including features such as geolocation-based recommendations, reviews, and sharing functionalities.

The admin manages accuracy and operational functionality of the system. POI data is managed by content managers and user-generated content (UGC) such as reviews and ratings are moderated. System administrators take care of organizational things (like database operations, geolocation services and API integrations), so, that app works solitude. All these efforts are managed by a management team that tracks the project's advancements made in reaching out to the wider range of audience by coordinating the marketing strategies. The clear distinction of these states aids in maintaining a smooth workflow, leading to a better user experience and reliable system behavior.

2.1.4 Organization Processes

Currently, the organization's processes for handling historical and cultural Points of Interest (POIs) are handled manually, without the benefit of system support. Researchers and field personnel that collect data regarding POIs — names, history, significance, location, events, etc. Interim leaders work with this information in an unstructured way with spreadsheets, notebooks, or other non-integrated file formats. There's no system to help you easily archive accessible records.

Data verification is performed after collection by subject-matter experts such as historians or cultural specialists. It includes scrutiny and cleansing of the raw information and validation of this process often demanding interaction between the scholars and the pros. The validated data then goes into either physical archives or common digital drives, which have no decent indexing or search functionality. When local tour guides, students or other stakeholders need information, staff must then search manually through these archives to locate the necessary details. Moreover, location-based recommendations are difficult to provide, and force you into general suggestions instead of precise geolocation, making users' requests more complicated.

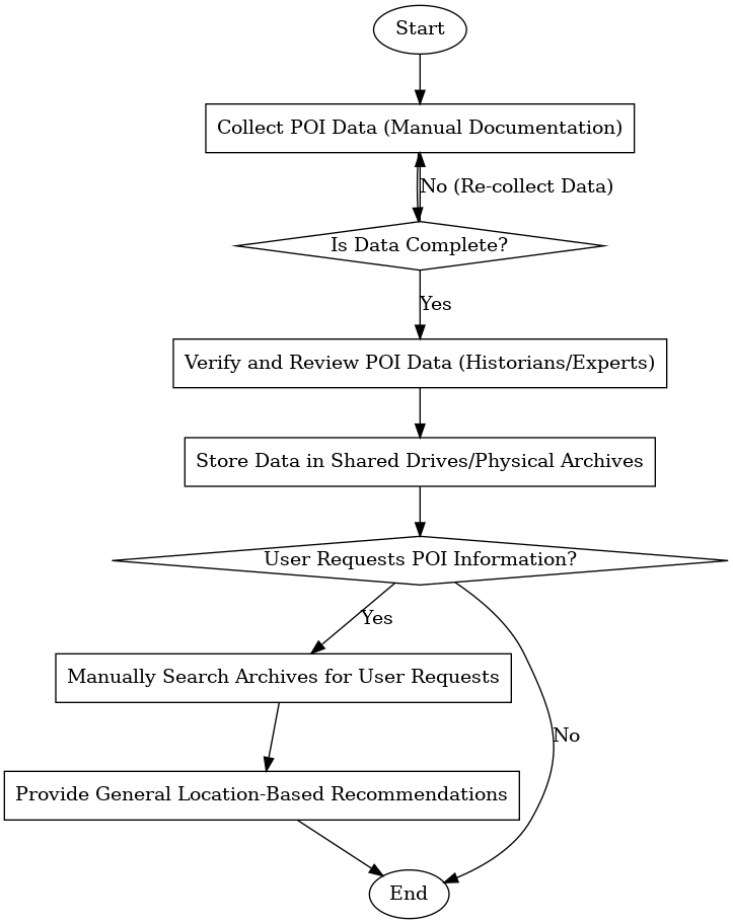


Figure 1: Activity Diagram for Current Organizational Processes

2.1.5 System Support in the organization

By automating and centralizing the manual processes of data collection, storage, and retrieval, the ReliveThePast system will revolutionize the entire process. To gather data, the system will use APIs and geolocation services based on GIS to fill POIs with precise and current data. Administrators will access a dashboard that enables them to validate and review the data compactly, avoiding the manual verification process we have to currently do — the back and forth. It also automates periodic reviews of high-priority POIs while ensuring the data is kept up to date at all times and never becomes stale.

The general concept of POIs also allows the system to intelligently assist users with POIs in real time through search, recommendations, and pushnotices based on the local context and preferences of the user. This will help serve queries much faster and in a far more efficient manner and allow users to have on-demand access to information through a mobile app. The location-based services will recommend relevant historical/cultural locations around them with which users can interact, making the experience more personalized. And, this smooth transition from data to user experience is a huge leap in efficiency and pleasure for the user.

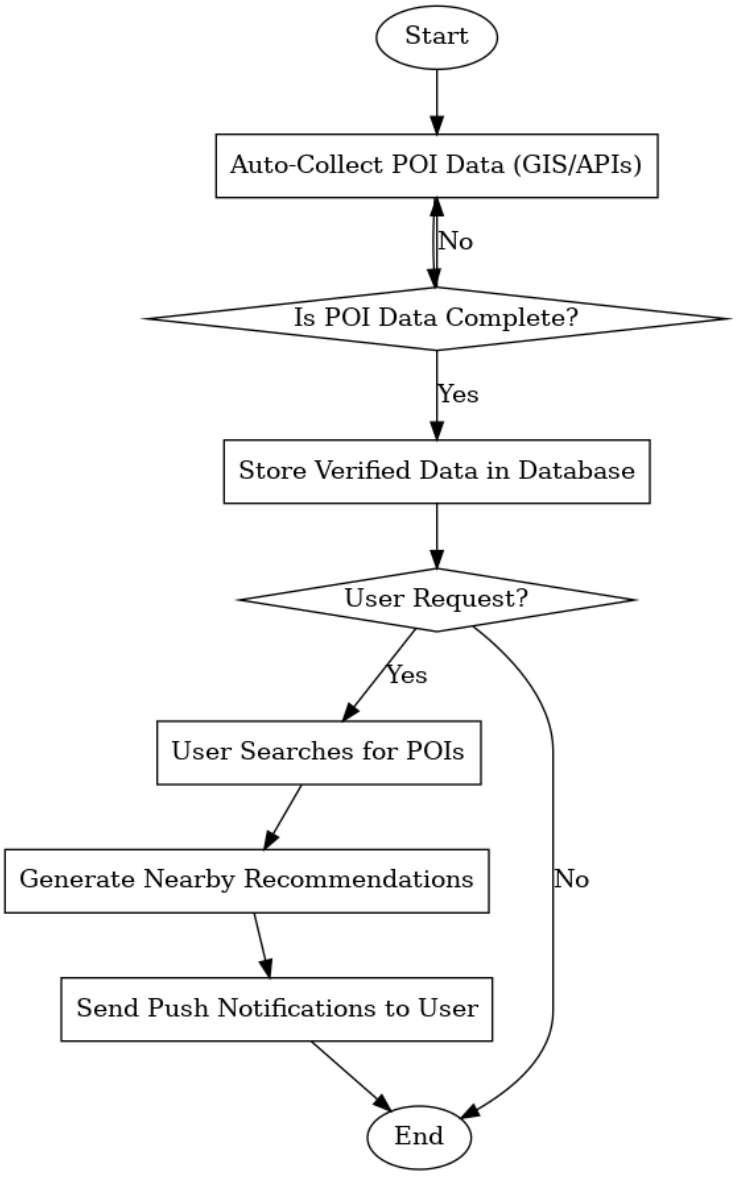


Figure 2: Activity Diagram for System Supported Processes

2.2 Generic Use Case Model

In Figure 3, the generic use case model for the system ReliveThePast is represented in the form of a package diagram. Each package aggregates one or more parts of the system designed to support organizational processes and/or group a set of functionalities. Each package includes examples of actors and use cases developed for the system.

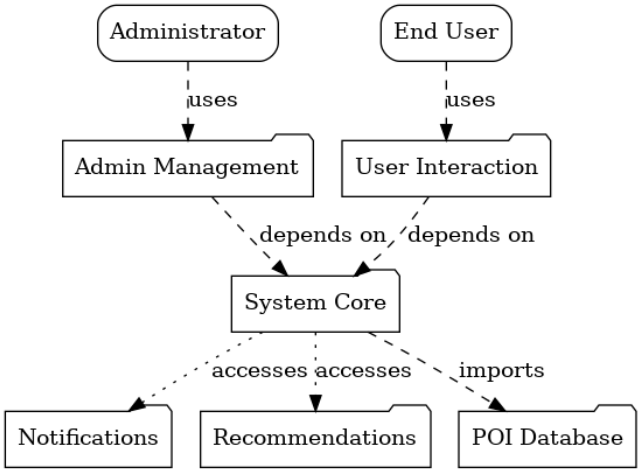


Figure 3: Package Diagram for RelieveThePast

Table 38 presents a description of each Actor in the system.

Actor	**Brief Description**
End User	The primary users of the system, including tourists, students, and cultural enthusiasts. They interact with the app to search for POIs, view details, save favorites, and receive location-based recommendations.
Administrator	System administrators responsible for managing POI data, verifying information, approving user reviews, and generating reports.

Table 38: Actors of the ReliveThePast System

Table 39 briefly describes each of the packages for the system ReliveThePast.

Package	**Brief Description**
User Interaction	Contains use cases that define the functionalities available to end users, such as searching for POIs, viewing details, saving favorites, and managing accounts.
Admin Management	Includes use cases for administrators, such as managing POI data, verifying information, and generating reports.
System Core	Provides the foundational features, such as geolocation services, which support the entire system.
Notifications	Manages the delivery of push notifications to users about nearby POIs and important updates.
Recommendations	Handles the generation of personalized recommendations for nearby POIs based on user preferences and location.
POI Database	Provides access to the centralized database storing all POI information, user preferences, and related metadata.

Table 39: Use Case Packages for the ReliveThePast System

2.3 Main exclusions

In Table 40, the processes not supported by the ReliveThePast system are presented. The justification for these exclusions relates to the fact that the system's scope is focused on delivering digital, location-based services, and some functionalities fall outside its primary objectives.

Process	**Brief Description**
Offline Access to Data	The system does not support accessing POI data or functionalities without an active internet connection. This is due to reliance on real-time geolocation and external APIs.
Physical Tour Guide Management	The system does not handle the booking or management of physical tour guides, as it focuses exclusively on digital content delivery.
Manual POI Data Collection	The system does not include functionalities for manually collecting POI data in the field; this is managed externally by researchers.
Real-Time Crowd Analysis	The system does not analyze or display real-time crowd levels at POIs due to the complexity of integrating external crowd data sources.

Table 40: Processes Not Supported in the ReliveThePast System

Use Case Diagram

Figure 4 illustrates the use case diagram for the package of the system ReliveThePast, providing a detailed view of the interactions between actors and system functionalities



Figure 4: Use Case Diagram for ReliveThePast

Non-functional and functional Requirements

4.1 Non-Functional Requirement

The ReliveThePast system must meet key non-functional requirements to ensure reliability and usability. Table 41 outlines these requirements.

Requirement	**Description**
Performance	The system should respond to user actions, such as searches or navigation, within some seconds under normal load.
Usability	The interface should be intuitive, requiring minimal training for new users and providing clear navigation and instructions.
Availability	The system should be operational most of the time, allowing uninterrupted access to users and administrators.
Security	User data should be stored securely
Maintainability	System updates, such as adding new POI data, should require minimal downtime

Table 41: Non-Functional Requirements

4.2 Functional Requirement

Requirement	Description
Search POIs	The system must allow users to search for points of interest (POIs) using keywords, or geolocation. Search results should be filtered by distance, popularity, or type.
View POI Details	Users must be able to access detailed information about a POI, including its historical significance, images, location on an interactive map, user ratings, and reviews.
Save POIs	Users should have the ability to save POIs to their personal list of favorites. Saved POIs must persist across user sessions and be synchronized with their account.
Rate and Review POIs	The system must allow users to submit ratings (e.g., 1-5 stars) and write reviews for POIs. These reviews should be moderated by administrators to maintain quality.
Manage User Accounts	Users must be able to create accounts, log in, update personal information, reset passwords, and manage preferences, such as notification settings and language options.
Admin POI Management	Administrators must be able to add, edit, or delete POIs.
Moderate Reviews	Administrators must be able to review, approve, or delete user-submitted ratings and reviews to ensure they fit in to the community guidelines.
Geolocation Services	The system must use geolocation APIs to identify the user's location and suggest nearby POIs dynamically. It should also allow users to view their route to a POI.

Table 42: Functional Requirements

Technologies

5.1 Technologies server-side

For the development of the "ReliveThePast" system, the selected server-side technologies aim to ensure scalability, seamless integration, and reliability. These tools and frameworks will support efficient backend operations and facilitate secure data handling, with added provisions for backups and version control.

5.1.1 Programming language

The app will use Kotlin as the programming language for the server-side development of the application. Kotlin is specifically engineered for Android development, delivering succinct syntax, advanced constructs such as coroutines for handling asynchronous tasks, and strong integration capabilities with Java libraries. Since QuestDB is an AntData, a KP, its use case in the Android ecosystem is popular due to compatibility, making it always viable with the tools and frameworks required in such projects.

5.1.2 Database Management

Firebase Realtime Database, a NoSQL cloud-hosted database, will be used for the project's database management. The Firebase Realtime Database synchronizes data in real-time across all connected devices seamlessly, which is important to the user experience, particularly when manipulating geolocation-based data. Flame store contains built-in features that integrate with Firebase Authentication and Google Cloud APIs, providing us with an easy and secure way to maintain the data of your application.

If the project ends up needing to store multimedia files or perform more complex database queries, then the second tier could involve Firestore or Cloud Storage.

5.1.3 Development Tools

The most used IDE will be Android Studio, which is the official integrated development environment for Android development. The Firebase SDKs can be easily integrated into our project by using Android Studio when we need to use features such as user authentication, database connection, push notifications, etc.

Firebase Functions will be used to support more backend functionality. These serverless, event-driven functions allow for the execution of backend logic, such as validating data, sending notifications, or managing more complex interactions.

5.1.4 APIs and Integration

- **Firebase Authentication API** It allows you to authenticate users so that they can sign up and log in to your app, along with password recovery functionalities.
- **Google Cloud APIs** These are helpful in all the backend-related features, enabling the usage of geolocation (Google Maps API), cloud storage, and cloud operation functionalities critical to offering location-based services.
- **Google ML Kit (Optional)** To implement more advanced AI features such as text recognition or image labeling.
- **Glide** `(com.github.bumptech.glide:glide:4.12.0)`
  A fast and efficient image loading library for Android. Used to load and display images from both local and remote sources efficiently. Supports caching, placeholders, and transformations.
- **Glide Compiler** `(com.github.bumptech.glide:compiler:4.12.0)`
  Annotation processor required for Glide's custom model loaders and image processing features.
- **uCrop** `(com.github.yalantis:ucrop:2.2.8)`
  An advanced image cropping library that allows users to crop and manipulate images before uploading. This was used in user-generated content workflows, such as customizing personal POIs with images.

5.1.5 Version Control and Backups

GitHub will be used for version control and backups to guarantee the safety and reliability of the codebase. By hosting the application source code on GitHub:

- Local machine failure does not lead to data loss; regular backups can be created.
- Versioning allows changes to be noted and collaboration; so updates or bug fixes can be undone if needed.
- GitHub Actions (CI/CD, automate tests and deployments, development workflow)

5.2 Technologies client-side

The main objective of client-side development for the "ReliveThePast" system is to provide an interactive interface that makes the interaction with the user seamless. You are focused on the technologies and tools related to designing and creating the visual and interactive parts of the application.

5.2.1 Programming Language and Layout Design

It will be used Kotlin to do the front-end logic and XML to create user interface (UI) layouts. XML is the standard for Android app design, offering a flexible and structured approach to defining the app's visual components. This enables developers to create responsive layouts that adjust to different screen sizes and orientations.

5.2.2 Development Tools

- **Android Studio** The main IDE (Integrated Development Environment) for Android development is Android Studio, which works with XML to build layouts and provides debugging tools, performance profiling, etc., for the frontend.
- **Material Design Components** The application will follow Material Design guidelines to maintain a consistent and visually appealing user experience, using pre-designed UI elements such as buttons, navigation menus, and cards.

5.2.3 Accessibility and Usability Standards

To ensure inclusivity and ease of use for all users, the application will adhere to the W3C Web Content Accessibility Guidelines (WCAG):

- **Accessibility** The app will feature proper contrast ratios, scalable fonts, and support for screen readers, making it accessible to users with visual or motor impairments.
- **Responsive Design** Using XML layouts will allow the app to perform efficiently on a diverse set of Android devices, from the smallest smartphones to the largest tablets.
- **Testing with Users** Testing with users (current or prospective) will ensure that its interface meets the needs of as many of its potential users as possible. Collect feedback to pinpoint pain points and create a better user experience.

Using Kotlin and XML for front-end development, ensuring compliance with accessibility standards, and doing iterative user testing will give the client-side of "ReliveThePast" a polished, user-friendly, and inclusive experience for its audience.

Logical and physical design

6.1 Logical organization of the system

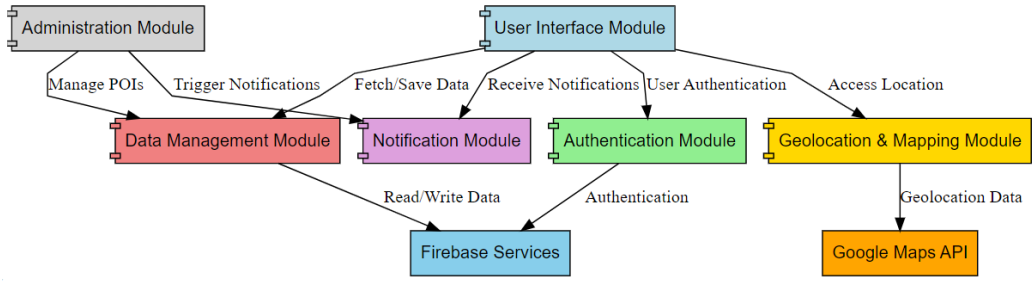


Figure 5: Logical organization of the system

The modular architecture of the "ReliveThePast" system has been chosen so the system is clear, maintainable, and scalable. Building modular software enables the specification of different functionalities to distinct modules that work together to form a system. Here is an overview of these major components and their functionality:

6.1.1 Main Components

User Interface Module:

- Interacts with the users and the system
- Handles visualization of data fetched from other modules, such as maps, places, and alerts.

Authentication Module

- Provides secure user authentication and authorization.
- Firebase integration to handle user login, signup, and session management.

Data Management Module:

- Responsible for storing, retrieving, and updating data from the Firebase Realtime Database.
- Acts as the central component for managing POI data, user preferences, and other system information.

Geolocation and Mapping Module:

- Handles geolocation using Google Maps API
- Real-time user location and POI recommendation based on user-home proximity.

Notification Module

- Targets users with location-based push alerts and status updates
- Manages notification delivery, using Firebase Cloud Messaging.

Administration Module

- Allows Admins to administer POI data — add, edit, and delete records.
- Includes tools for moderating user-generated content and firing notifications.

6.1.2 External Modules

Firebase Services:

- Offers backend features such as real-time data storage, authentication, and cloud messaging.

Google Maps API:

- Enables geolocation and mapping capabilities for displaying POI and end-user locations.

6.1.3 Component Relationships

As the modular components interact with each other, it delivers a unified experience to the users. The User Interface Module interacts with the Authentication, Geolocation and Mapping, Notification, and Data Management Modules to retrieve and display data for users, facilitating seamless user interactions. Authentication Module This module is responsible for integrating with Firebase Services and securely handling user verification and its session management. The Real-time Location and POI Recommendation sub-module of the Geolocation and Mapping Module communicates with the Google Maps API. Likewise, the Administration Module integrates with the Data Management Module to manage POI data and communicate with the Notification Module to send system updates. Such interconnected structure provides efficient, reliable, and maintainable system architecture.

6.2 Physical organization of the system

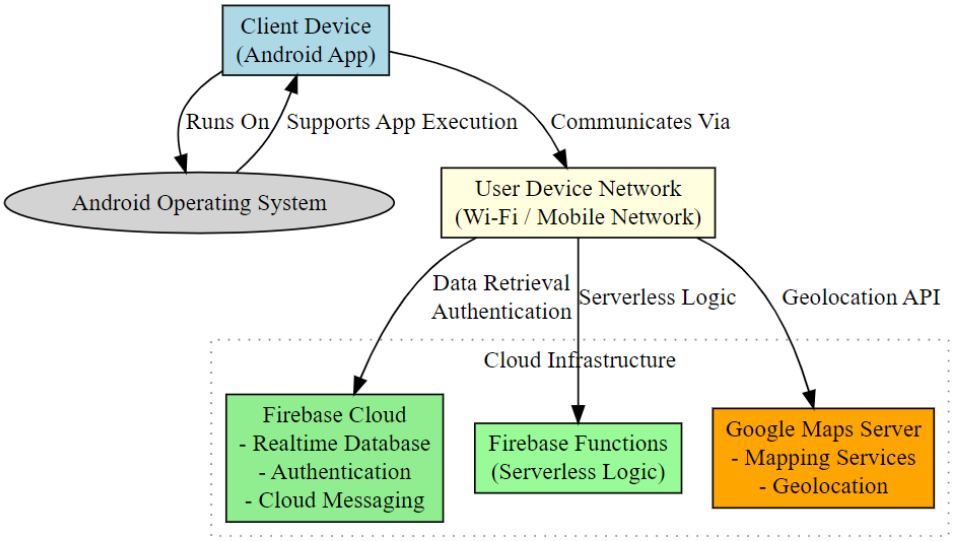


Figure 6: Physical organization of the system

6.2.1 Key Components

Client Device (Android App):

- This represents the user's mobile device, where the application is installed and executed.
- It handles user interactions and displays data retrieved from the backend systems.

Android Operating System:

- Represents the mobile operating system (e.g., Android) running on the client device.
- It supports the execution of the application by managing resources, permissions, and hardware access.

User Device Network (Wi-Fi / Mobile Network):

- Represents the communication layer connecting the client device to cloud-based services.

- The network ensures data flows securely between the client app and external systems using standard protocols like HTTPS.

Firebase Cloud:

- **Realtime Database:**Stores and synchronizes data in real time between the app and the cloud.
- **Authentication:**Manages user accounts, login sessions, and identity verification.
- **Cloud Messaging:** Delivers push notifications to the client device.

Firebase Functions:

- Represents serverless backend logic hosted in the cloud.
- Handles event-driven processes such as validating data, generating notifications, or processing user requests that require custom backend functionality.

Google Maps Server

- Provides geolocation and mapping services.
- The app interacts with this external service to display maps, calculate routes, and recommend nearby points of interest (POIs).

6.2.2 Relationships and Interactions

The components of the "ReliveThePast" system interact seamlessly to ensure functionality. The Client Device runs the Android app on the Android Operating System, which manages resources and hardware. The app communicates with the Firebase Cloud via the User Device Network (Wi-Fi or mobile data) for real-time data synchronization, user authentication, and push notifications. It also utilizes Firebase Functions for server-side logic like data validation. Additionally, the app interacts with the Google Maps Server to fetch geolocation and mapping data, enabling features like map displays and POI recommendations. Together, these components ensure secure and efficient communication for a seamless user experience.

6.3 Database schema

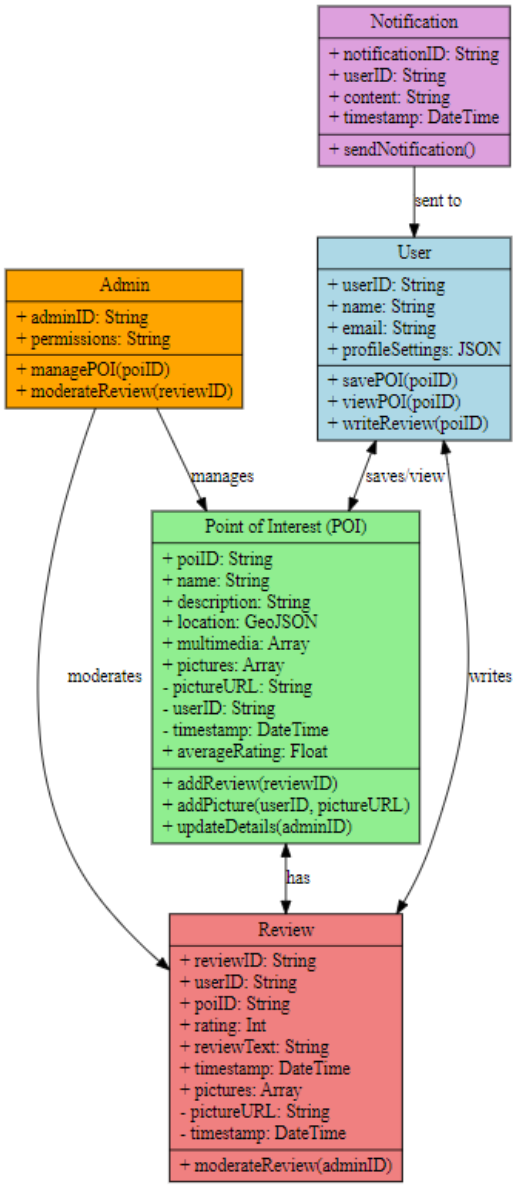


Figure 7: Database Schema

6.3.1 Entities

User:

This entity saves information for all users who are registered in the "ReliveThePast" system. Every user record is uniquely identified and also contains important information about the user, such as name, email, and profileSettings (for example, user language or notification settings). The entity allows users to make system interactions such as saving POIs (points of interest), viewing specific POI information, and writing reviews. These techniques enhance the user's experience and enable users to interact with the application in a meaningful way.

Point of Interest (POI):

POI entity stands for Points of Interest, which represents the landmarks in the system (cultural or historical ones). Each POI has a unique identifier (poiID) and consists of attributes including name, description, location (latitude and longitude), and multimedia (general content like images or videos). It also has a pictures array to store user-uploaded images, alongside metadata like userID (who uploaded it) and timestamp. Represents the core of user interaction, holds supplementary information such as averageRating, which is calculated from reviews, and allows updating admin side info via the updateDetails method.

Review:

The Review entity represents the feedback given by the users on certain POIs. The reviewID uniquely identifies each review, tied to the userID and poiID. The review has a rating (numerical), the meat of the review (text), and a timestamp for when each was created. This optionally allows users to upload pictures associated with the review through a pictures array. They are a community-driven content engine that provides rich POI context, whilst ensuring quality content moderation via the admin's moderateReview method.

Notification:

This is used to handle messages created by the system to be sent to the user to increase engagement and provide them with useful information. A notification is individually identified by notificationID and connected with userID. These also consist of content for the notification message and a timestamp for when it was sent. The sendNotification method ensures timely delivery, enabling users to stay updated on nearby POIs and system updates or customized reminders.

Admin:

System management and moderation tasks are handled by the Admin entity. Each of these admins is uniquely identified by an adminID, which is associated with a permissions field defining their access level, e.g., able to update POIs, able to moderate reviews. Administrators utilize functions such as manage-POI for editing or deleting POIs and appreciate the moderation feature for user-generated content through moderateReview, making them contribute appropriately to the overall system policy. This entity keeps the system operable, providing correctness and inappropriateness control.

6.3.2 Relationship

The "ReliveThePast" system as a whole ensures smooth interaction between individual entities within a complex and efficient database structure. POIs are a way for users to search and save points of interest; patients interact through methods like savePOI and viewPOI, allowing a personalized experience. In addition, they provide valuable community-based information through writing POI reviews, creating a relationship between User and Review, and then, since reviews reference POIs, we create a one-to-many relationship that enriches information about POIs. They can maintain and update the system by adding or modifying points of interest (POIs) and moderating reviews to maintain the integrity and accuracy of the data through methods such as managePOI and moderateReview. Moreover, notifications are user-oriented, allowing personalized communication to be sent through the sendNotification method to enhance the overall user experience. These relationships facilitate the system's scalability and usability while also assisting in keeping the system efficient.

Solution Developed

7.1 Introduction

The solution developed, ReliveThePast, is a mobile application that combines geolocation, multimedia content, and user interactivity to provide a dynamic cultural exploration experience. Users can discover nearby points of interest (POIs), view detailed information including images and historical context, save favorites, rate and review sites, and even create their own personal POIs. These features are designed to make cultural engagement more accessible, spontaneous, and personalized. Administrators are equipped with tools to manage official POI data, moderate user reviews, ensuring data quality and continuous system improvement.

Compared to the alternative solutions analyzed during the benchmarking process, ReliveThePast stands out for its user-centered design and interactive capabilities. While other apps often offer static information or require predefined tour paths, this solution supports real-time exploration and user contribution, making it more flexible and engaging. The use of open-source technologies and cloud-based tools like Firebase, Glide, and uCrop ensures a scalable, modern backend infrastructure that supports a smooth and responsive user experience. This approach not only improves upon the limitations identified in existing apps but also introduces new possibilities for learning, discovery, and cultural preservation.

The full source code and development history of the application can be accessed at: github.com/JoseMachado21/RelieveThePast

The latest APK version of the application can be downloaded here: Download APK

7.2 Architecture

The architecture of ReliveThePast is based on a modular, client-server design that ensures scalability, maintainability, and performance. The solution is composed of multiple integrated components distributed across client-side and server-side layers, leveraging cloud services to support real-time functionality and responsive user experiences.

The following diagram illustrates the architecture of the developed system:

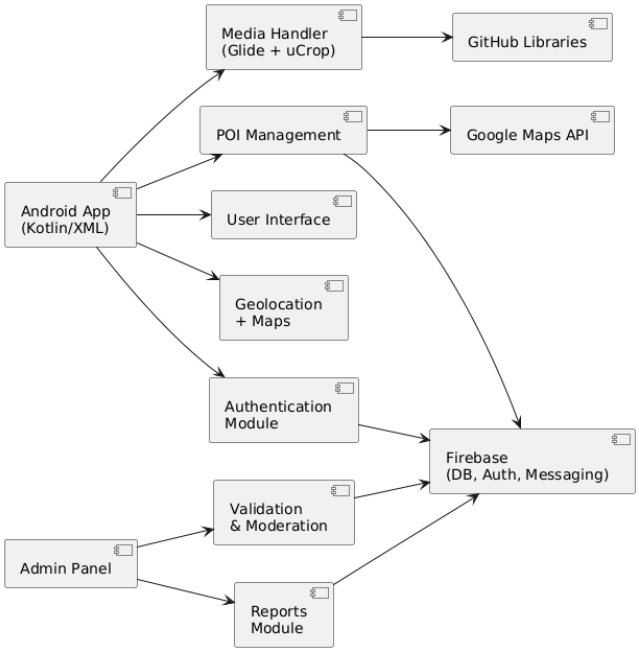


Figure 8: Components Diagram

7.2.1 Main Components

- **Android App (Kotlin/XML):** The client-side application developed in Kotlin, responsible for UI rendering and handling user interactions, following Material Design guidelines.
- **User Interface:** Manages navigation and layout, serving as the main access point to all features.
- **Authentication Module:** Handles user login and registration, integrated with Firebase Authentication for secure access.
- **Geolocation and Maps:** Provides map-based POI exploration using the Google Maps API and the device's location services.
- **POI Management:** Enables users to view, create, and manage Points of Interest (POIs) with real-time updates via Firebase Realtime Database.
- **Media Handler (Glide + uCrop):** Integrates external libraries to load and crop images, enhancing multimedia interactions.
- **Firebase Backend:** Offers cloud-based storage, real-time data syncing, authentication, and messaging services.
- **Admin Panel:** Provides administrative tools for content moderation, POI management, and usage reporting.
- **Validation & Reporting Modules:** Support backend moderation and generation of usage reports to ensure content quality and system insights.
- **External Libraries (GitHub):** Open-source tools like Glide and uCrop extend app functionality with minimal development overhead.

7.2.2 Architectural Choices

The architecture was created with a modular and scalable concept to accommodate a mobile-first application focusing on usability, speed, and adaptability. By separating features into dedicated resources (ajax endpoint, ui framework, authentication, POI management, geolocation, etc.), it would be easier to maintain, test and evolve.We chose Firebase for the backend to leverage its strong support of Android, real-time sync and out-of-the-box authentication and messaging. Kotlin was used for its modern syntax, conciseness, and because it is fully interoperable with Android development.

Combining the usage of Glide and uCrop as GitHub added up to a good mix of best benefits of both worlds: leverage the same established, community-driven tools to enhance main feature set while still keeping development overhead low.This architecture also paves the way for more features in the future, be it a web frontend, ML-based POIs suggestions or even AR experiences, without needing to modify the foundational principles of the system.

7.3 Technologies and Tools Used

The development of ReliveThePast was guided by the goal of building a scalable, user-friendly, and interactive mobile solution that could respond in real time to user input and geolocation. To achieve this, a carefully selected stack of technologies and development tools was employed, each chosen for its reliability, integration potential, and relevance to the project's functional and non-functional requirements.

This section outlines the main tools and technologies utilized throughout the project, along with a brief description of their roles and the rationale behind their selection.

7.3.1 Tools and Platforms

- **Android Studio**
  Primary development environment for the mobile application.
  Chosen for its full support of Kotlin, seamless integration with Firebase, and powerful debugging and UI design tools.

- **Firebase Console**
  Used to manage backend services including database, authentication, and push notifications.
  Its real-time capabilities and ease of use made it an ideal choice for rapid backend integration without the need for a custom server.

- **Git & GitHub**
  Version control system and collaborative development platform.
  Enabled efficient code tracking, branching, and collaboration, especially important during iterative development phases.

- **PlantUML / WebGraphviz**
  Diagramming tools used to model system architecture and use cases.
  Provided a simple, text-based way to produce professional UML diagrams for documentation and planning.

- **Figma**
  UI/UX design tool used to create high-fidelity wireframes and interface mockups.
  Allowed for rapid prototyping and visual alignment of the app's design with user experience goals before development began.

- **Glide (GitHub)**
  Efficient image loading and caching library integrated into the app.
  Ensured smooth media handling without custom image management logic.

- **uCrop (GitHub)**
  Image cropping library for allowing user-controlled image editing.
  Enhanced the personal POI creation experience by letting users upload and edit images within the app.

- **Gradle**
  Build automation system used to manage dependencies and compile the Android app.
  Simplified project configuration and ensured consistent builds across environments.

7.4 Scope of Curricular Application

The development of ReliveThePast benefited from the integration of knowledge acquired across multiple curricular units in the degree program. While all subjects contributed in meaningful ways to the formation of technical and methodological competencies, several played a particularly direct role in shaping the final solution.

- **Interactive Systems Design**
  Provided essential foundations for conducting usability evaluations, such as the System Usability Scale (SUS), and informed the development of mockups and interface prototypes during the early stages of design.
- **Systems Analysis and Design** and **Software Engineering**
  Guided the use of UML and architectural diagrams, many of which were introduced and practiced during these courses. Additionally, the document structures shared in these units greatly influenced the layout and organization of this report.
- **Project Management**
  Contributed with valuable strategies for planning and execution, particularly in the use of Gantt charts and project tracking tools, ensuring that the development followed a structured and realistic timeline.
- **Mobile Device Programming**
  Enabled the practical understanding of Android Studio and Kotlin, which were fundamental to the implementation of the mobile application.
- **Human-Computer Interaction**
  Enriched the development process with considerations around usability and accessibility, which are being applied during the testing phase to ensure a more inclusive and user-friendly experience.

These academic contributions demonstrate how the project represents a culmination of interdisciplinary knowledge, effectively applied to solve a real-world problem through software development.

7.5 Components

ReliveThePast is built from multiple important technical components that put together, make the functionality, performance and user experience we provide. Every piece was chosen for its ability to make development easier, for its increased interactivity, and to provide what was needed functionally from the ground up

7.5.1 Firebase Realtime Database

In 2017, a NoSQL based database system developed by the Firebase team was added in the latest version. This is the main data store of the application. It makes it possible to synchronize the app with the cloud in real time, keeping the POIs, users (their saved places, obeservations) and internal operations updated for the network of connected devices. The JSON-based format and offline support made for a fast and stable user experience.

7.5.2 Google Maps API

Part of this includes receiving data from the Google Maps API to render maps and track the user's geolocation in order to present nearby POIs on their current map view. It can handle map overlays, custom markers, and dynamic navigation to/from POIs, making itself part of the geographic heart of the user experience.

7.5.3 Google Places API

In combination with the Maps API, the Places API facilitates the creation of POIs. It allows features such as autocomplete place names, getting standardized data (coordinates and addresses) and improving accuracy of custom POI entities added by users.

7.5.4 Glide (GitHub)

Glide is an open-source image loading library that is used to load and display images of POI around the app. Sometimes you just want an image loader that can perform the simple and most common image loading tasks very quickly and very memory efficiently without all the complications or the complete re-write. An Image loading library that takes care of the most common use cases you'll run across including: fast and small image loading, simple to use API, high-quality, quick image transformations, automatic caching, logging and telling the developer what's going on. And for slower connection scenarios or users that have limited device hardware, it stays consistent and offers the ability to use placeholders to keep a uniform look and feel for your app.

7.5.5 uCrop (GitHub)

uCrop is used to crop and scale images by users when uploading photos for personal POIs. Its incorporation enhances content quality and enables users to have some say in how their uploaded media is displayed within the app. This add-on is small and quickly integrated into image-input flow.

7.6 Interfaces

The user interface of *ReliveThePast* was designed with clarity, simplicity, and responsiveness in mind. The application follows Android's Material Design principles and was implemented using Kotlin and XML layout files. Below are some of the key screens involved in the user authentication and onboarding process.

7.6.1 Welcome, Login, and Sign-Up Screens

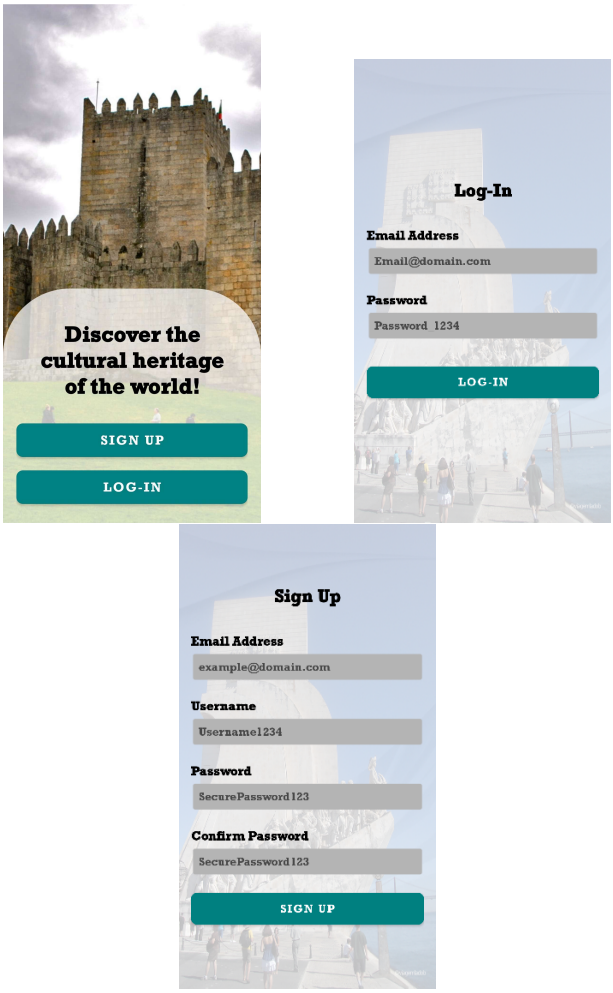


Figure 9: Welcome Screen, Sign-Up Screen , and Login Screen

The Welcome screen serves as the initial entry point. If a user is already authenticated, the app skips directly to the main interface. Otherwise, users can choose to log in or sign up. The screen includes animations such as `fade_in` and `slide_up` for a smooth, engaging introduction.

Existing users can Sign In using their email address and password. Validation is done using Firebase Realtime Database. If the credentials are valid, email and user's username are saved locally into `SharedPreferences`, which enables session storing. Invalid entries receive error feedback and the visual properties of the UI design increase the visibility with a transparence effect on the background image.

Located on the Sign-Up screen, new users enter an email, username, password, and confirm password. Password integrity is enforced (minimum length, one upper case, and a number) and the app checks if the email is the primary in firebase before registering. And if everything is okay then we securely store the users data in Firebase such that it is saved under a unique auto-generated id. The form is well designed and validation logic is geared to make the onboarding experience a smooth and frustration less experience.

7.6.2 Main Activity and Map Interface

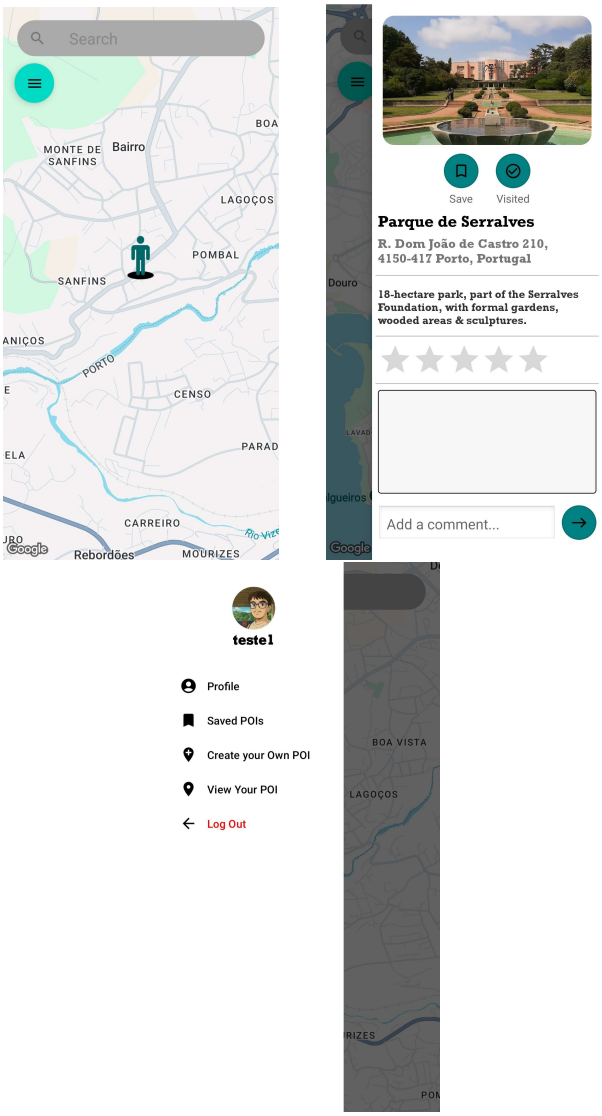


Figure 10: Main Map View, POI Details View, Navigation Drawer

The **Main Activity** serves as the core of the application after successful login, combining several key functionalities: a real-time map interface using the `Google Maps API`, a dynamic POI detail drawer, and a navigational drawer offering access to additional features.

The user's current location is retrieved using `FusedLocationProviderClient` and displayed with a custom marker. Permissions are handled at runtime, and the app automatically begins listening for location updates when active. The Google Places API powers the top search bar with location autocomplete and dynamic map movement.

When a POI is tapped, a right-side drawer appears with detailed information including name, description, address, image (fetched via Places API), and interactive options: mark as visited, save to favorites, rate, and comment. Comments and ratings are stored per user under a unique POI path in `Firebase Realtime Database`, and are dynamically loaded along with user profile pictures.

A side navigation drawer provides access to user profile, saved POIs, personal POI creation, and logout. The user's profile info is fetched from Firebase and displayed using locally stored session data via `SharedPreferences`.

Significant technical considerations include:

- **Custom location marker**: replaces default map pin for better UX.
- **Dynamic drawer loading**: enhances flexibility and separation of logic.
- **Firebase integration**: handles real-time storage and retrieval of user-specific actions like saving, visiting, and commenting.
- **Comment moderation**: includes logic for deleting one's own comments, triggered by adapter callbacks.

7.6.3 Profile Screen

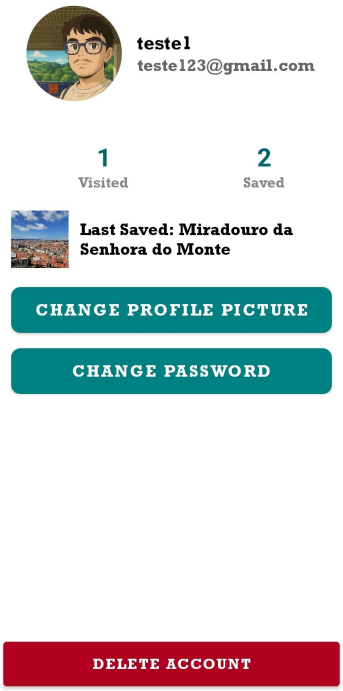


Figure 11: Profile Screen – User Overview and Account Management

The **Profile Screen** provides users with an overview of their account and key activity metrics. It displays the user's profile image, username, email, the total number of visited and saved POIs, and the most recently saved POI, which includes a title and thumbnail image. This screen also provides access to account management actions such as changing the profile picture, updating the password, and permanently deleting the account.

All data is retrieved from the `Firebase Realtime Database`, and user session information is maintained locally using `SharedPreferences`. The user's last saved POI is determined by the most recent `savedAt` timestamp, and the place's name and photo are fetched dynamically using the `Google Places API` and `FetchPhotoRequest`.

Key technical highlights include:

- Use of **Base64 image decoding** to render profile pictures stored in Firebase.
- Implementation of a **custom account deletion workflow**, which removes all user-related data including saved, visited, and created POIs.
- Real-time metric updates for POI counts, ensuring the information reflects the latest user activity.
- Dynamic image loading from Google Places API for the latest saved POI.

7.6.4 Create Personal POI Screen

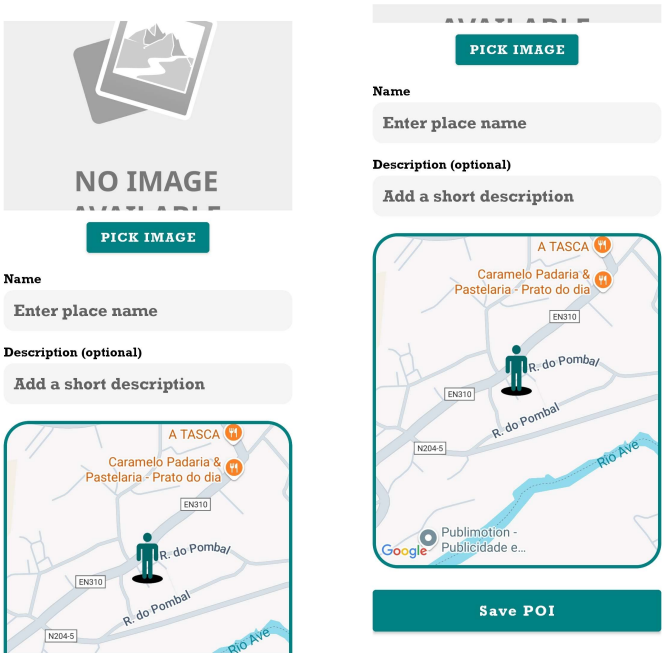


Figure 12: Create POI Interface – Image Selection, Description and Map Pinning

The **Create Personal POI** screen allows users to contribute personalized points of interest to their private collection. The interface guides users through uploading an image, providing a title and optional description, and selecting a location directly on the map.

On launch, the map automatically centers on the user's current location using `FusedLocationProviderClient`, and users can manually drop a marker to tag the desired POI location. Image selection is enabled via the Android system gallery, and the chosen image is previewed on screen. Once submitted, data is encoded and saved to Firebase under the `CreatedPOI` node, uniquely linked to the user's ID.

Key implementation features include:

- **Custom marker rendering:** User and POI markers are drawn using a scaled bitmap with optional color overlays.
- **Image handling:** Images are compressed and Base64-encoded before being uploaded to Firebase, optimizing storage.
- **User-specific Firebase paths:** Each POI is stored under the logged-in user's UID, retrieved through email verification in Realtime Database.
- **Input validation:** Ensures both a title and geolocation are provided before submission. A default value of "NoImage" is stored if the user skips the image.

7.6.5 Saved POIs Screen

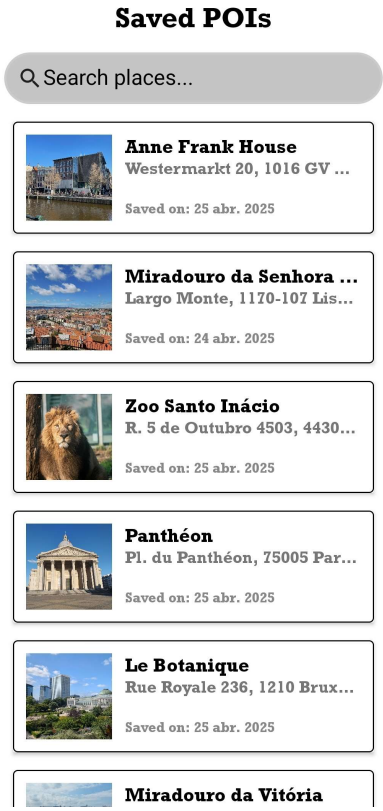


Figure 13: Saved POIs Interface – List of Personalized Favorites

The **Saved POIs** screen offers users quick access to locations they've previously bookmarked. Each entry displays a title, address, image preview, and the date of saving. The design promotes readability through card-styled entries and a search bar for filtering saved places in real time.

Data is loaded from Firebase's `SavedPOI` node using the user's ID retrieved via local shared preferences. Once retrieved, each saved POI is enriched with data from the Google Places API (name and address). Images are dynamically loaded when available, and the RecyclerView adapter is refreshed accordingly.

Implementation highlights include:

- **Dynamic search filtering:** A text watcher filters the RecyclerView based on name/address.
- **Google Places API integration:** Used to fetch POI metadata and ensure consistency with map content.
- **Clean back-navigation flow:** Selecting a POI sends its Place ID back to `MainActivity` for focused display.
- **Empty state UI:** A fallback layout appears when the user has no saved places, improving user feedback.

Testing and Validation

8.1 Introduction

A sound testing and validation strategy had been applied to the constructed solution to make sure that it complies with the functional and non-functional scope. This method consisting of several layers of testing to ensure accuracy, reliability, and user-focused capabilities. Unit tests covered isolated logic and individual functions including form validation and field formatting. Integration tests for testing how modules work together like the interaction of the client-side app with the Firebase backend services. Functional tests verified all features – login, POI interaction and account handling worked as intended under the different possible inputs. Usability testing was also conducted to assess the interface and user experience using things such as task-based walkthroughs and the System Usability Scale (SUS) to test for ease of use while noting areas that could be improved.

The fact that the validation was performed at different levels of the architecture allowed not only confirming the applicative works as expected but identifying how the applicative actually fulfils its main goal, enhancing communication and discovery of points of interest (POIs) meaningfully and user-friendly.

8.2 Unit Tests

The correctness of individual methods and parts of small components (in the area of data validation and core logic) was tested with unit testing. These tests were particularly important to verify edge cases, and to ensure invalid input could be processed without having to execute the whole system.

Critical tasks identified as sensitive for unit testing were:

- Format validation for the email address, password strength, and required fields in the login and signup process.
- Field comparison routines, such as password and confirm password matching.
- Helper functions like Firebase image encoding in Base64 and data formatting for storage in Firebase.

Because of the Android-specific focus of the app, unit tests were conducted with **JUnit** in its Android context. Mocked dependencies allowed simulating interactions with Firebase when required, making the test boundaries explicit.

These tests also helped catch issues earlier in the development cycle, supporting confidence in the reliability of shared elements that are reused across many different screens or features. The independent validation of these fundamental functionalities greatly enhanced the system's reliability and sustainability.

8.3 Integration Tests

Integration testing was performed to verify the correct interaction between individual modules and third-party services within the application. Unlike unit tests, which focus on isolated components, integration tests validate how different parts of the system work together in a cohesive flow.

Given the nature of the application, special attention was given to the integration between the Android frontend and external services such as:

- **Firebase Realtime Database:** Testing whether user data (login, POIs, comments, ratings) was correctly saved, updated, and retrieved in real time.
- **Google Maps API:** Ensuring correct rendering of maps, markers, and user location tracking.
- **Google Places API:** Verifying that POI metadata such as names, addresses, and photos were correctly fetched and displayed.
- **Image handling and encoding:** Testing the correct flow from image selection, cropping, Base64 conversion, and storage in Firebase.

These tests were often executed manually in combination with debug logging and database monitoring, ensuring that end-to-end data flow was working as expected. For example, when a user creates a POI, tests ensured that:

- The POI is stored under the correct user ID in Firebase.
- The geolocation marker appears correctly on the map.
- Encoded image data loads properly on future views.

Although automated integration tests were limited due to dependencies on cloud services, real-device and emulator-based testing played a critical role in validating multi-component workflows across different usage scenarios.

8.4 Functional Tests

Functional testing was performed to ensure the application features behave as expected and are based according to the proposed requirements. These tests were intended to resemble real-world usage and include both normal and more extraordinary paths through the core features.

I performed thorough testing of each big user-facing feature, such as:

- **User Authentication:** Verifying that login, signup, and logout functionalities handle valid and invalid inputs correctly, and that sessions are maintained using SharedPreferences.
- **POI Interaction:** Ensuring users can search, view, save, rate, comment on, and mark POIs as visited. The proper updating and retrieval of this data from Firebase was validated in each case.
- **Create/Edit Personal POIs:** Testing that users can create custom POIs with geolocation, descriptions, and optional images. Field validation, image encoding, and Firebase storage were all covered.
- **Profile Management:** Validating profile updates such as password changes, profile image uploads, and account deletion, ensuring correct backend updates and immediate interface feedback.
- **Navigation and Map Interaction:** Confirming the correct functioning of map navigation, drawer menu links, and dynamic UI elements (e.g., opening the right-side detail drawer when a POI is tapped).

The functional tests were primarily executed on real physical Android devices and for emulators, thus enabling the validation of applications across different screen sizes, permissions, and network conditions. All the tests indicated that the application achieved its primary goal: enabling the users to navigate, interact, and manage Points of Interest in an intuitive and reliable fashion.

8.5 Usability Tests

Usability evaluation was an important step to confirm the user experience and to ensure that the interface was simple, and easy-to-use, and similar to the regular use for the end-users. The purpose of these evaluations wasn't just to verify that users could accomplish tasks, but also assess how easily and productively they could accomplish tasks

Experiments have been performed with a small number of users covering subjects with different usage profiles: users that know or do not know mobile map-based applications. For these tasks, participants were requested to perform specific actions, such as creating a personal POI, saving a location, updating their public profile picture, and switching from one main area of the app to another.

Throughout the tests, several metrics were considered:

- **Task success rate:** Whether users were able to complete each task without assistance.
- **Time on task:** The duration taken to complete specific workflows.
- **Error rate:** Frequency and type of mistakes made (e.g., input errors, navigation confusion).
- **User satisfaction:** Collected using the **System Usability Scale (SUS)**, providing a quantifiable score based on user perception.

8.5.1 System Usability Scale (SUS)

For assessing the perceived usability of the app, **System Usability Scale (SUS)** was used as a standard and well-founded approach. The SUS is a set of 10-question in which participants are asked to rate and then provide brief comments on a 5-point Likert scale from "Strongly Disagree" to "Strongly Agree," intended for the overall assessment of the usability from the perspective of the user.

At the end of each usability testing the questionnaire will be administered. Participants will be invited to use the application in real or simulated use conditions and when they finish their tasks, they will anonymously complete the SUS. Both tech users and non-experts are targeted with this audience-selection approach in order to ensure a broad range of user experiences is captured in the feedback.

The standard SUS questions to be used are:

1. I think that I would like to use this app frequently.
2. I found the app unnecessarily complex.
3. I thought the app was easy to use.
4. I think that I would need the support of a technical person to use this app.
5. I found the various functions in this app were well integrated.
6. I thought there was too much inconsistency in this app.
7. I would imagine that most people would learn to use this app very quickly.
8. I found the app very cumbersome to use.
9. I felt very confident using the app.
10. I needed to learn a lot of things before I could get going with this app.

The responses will be scored according to the SUS scoring method and averaged to generate a usability score out of 100. This score will be used to support conclusions about the overall user experience quality and to identify areas for improvement.

8.5.2 NPS (Net Promoter Score)

To evaluate users' perception of ReliveThePast, I followed the user evaluation method of using the Net Promoter Score (NPS) to measure user satisfaction and willingness to recommend the technology to others. The NPS metric is a common measure in software and service assessment that measures user loyalty and general product feeling.

The NPS questionnaire asked users one primary question: "On a scale from 0 to 10, how likely are you to recommend ReliveThePast to a friend or colleague?"

- Promoters (score 9–10): Loyal fans who will promote the app.
- Passives (score 7–8): Satisfied but unenthusiastic users.
- Detractors (score 0-6): Unhappy customers who have the potential to dissuade others.

The NPS is then calculated by subtracting the percentage of Detractors from the percentage of Promoters. This yields a score between -100 and +100, with higher values indicating stronger user loyalty and satisfaction.

8.5.3 Customer Satisfaction Score (CSAT)

I used the same approach to elicit feedback (Feedback Prompt) from the users when they do or after they do something using ReliveThePast and also used the Customer Satisfaction Score (CSAT) method to assess how the users feel about their immediate interaction with the app. The CSAT score is a simple but powerful metric which is often used in software usability and customer support testing.

An example of a CSAT question might be something like: "Some user are asked specific questions to rate their satisfaction shortly after a critical event, such as: "How happy are you with how this app has worked?" Respondents are then asked to rate their satisfaction on a scale (usually ranging from 1(very dissatisfied) to 5(very satisfied)) with variations in the scale structure according to the specific e valuation context.

CSAT is measured by dividing the number of users who gave the top scores (e.g., 4 or 5) by the total number of ratings, and multiplying by 100 to get a percentage score.

Method and Planning

This is the Gantt chart of the acronym "ReliveThePast" indicating the time that our project is carried out between September 2024 and July 2025. It starts with Requirements Specification (Sep–Nov 2024), then there's Software Engineering Report (Nov 2024–Jan 2025) and Database Implementation (Nov 2024–Feb 2025). Application Implementation will occur from February through July 2025, with Front-End and Back-End Development from February to April. Testing phases are: Unitary Tests: Apr–May 2025; Search for Errors: May–Jun 2025; Error Correction: mid-May–mid-Jun 2025. The project ends with the Final Document (Jun–Jul 2025), which solidifies a structured route to completion.

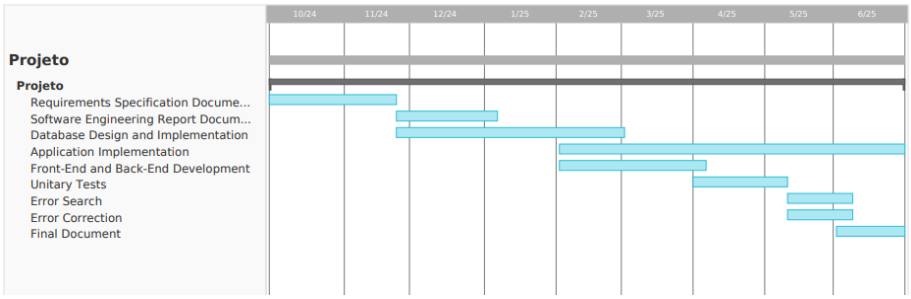


Figure 14: Gantt Diagram

Results

10.1 Test Results

10.1.1 Usability Tests

A structured user test of ReliveThePast was performed to ensure usability and user experience with the application using 9 participants. Objective is to get feedback on how actual users are using the application, and to catch any thing, like adding design elements etc. before final cut-over.

For our (rapid) usability evaluation participants had to perform a set of five key tasks which were interpreted to correspond to typical user interactions with the app. These tasks were creating an account, navigating the app interface, searching for POIs, saving a POI, reviewing a POI and accessing their saved locations. These activities represent representative use cases, and cover the most critical features of the app.

Each participant successfully carried out the five tasks and so the system has been found to be very usable and intuitive from the interaction design point of view. General feedback from the sessions indicated users found layout clear, navigation smooth, and accessed features in logical manner. Although there were small recommendations for visual and performance improvement, the general conclusion was that the application is user friendly and functionally accessible to the target group at this time.

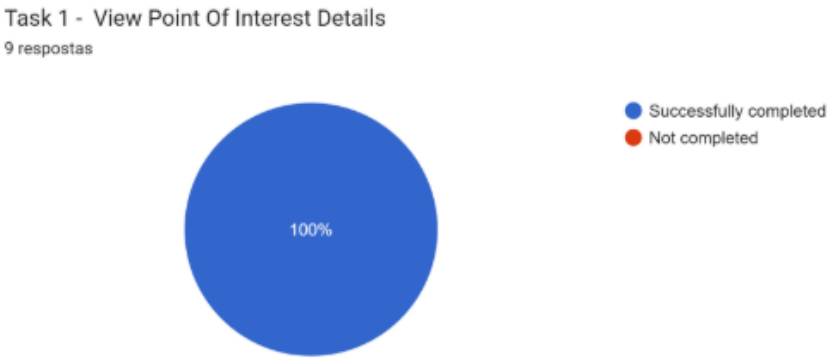


Figure 15: Task 1-View Point Of Interest Details

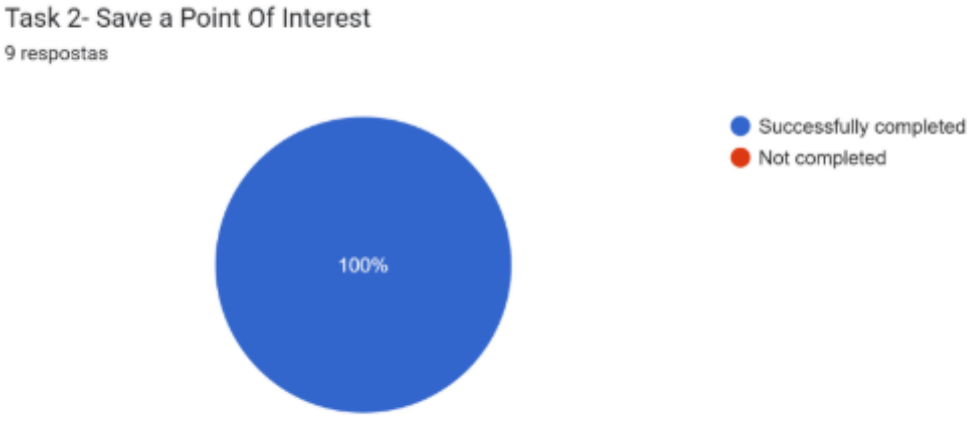


Figure 16: Task 2-Save a Point Of Interest

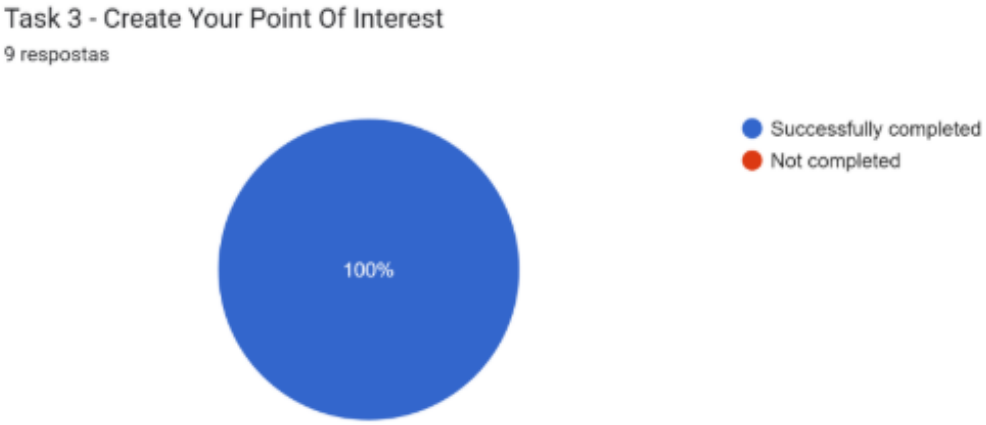


Figure 17: Task 1-View Point Of Interest Details

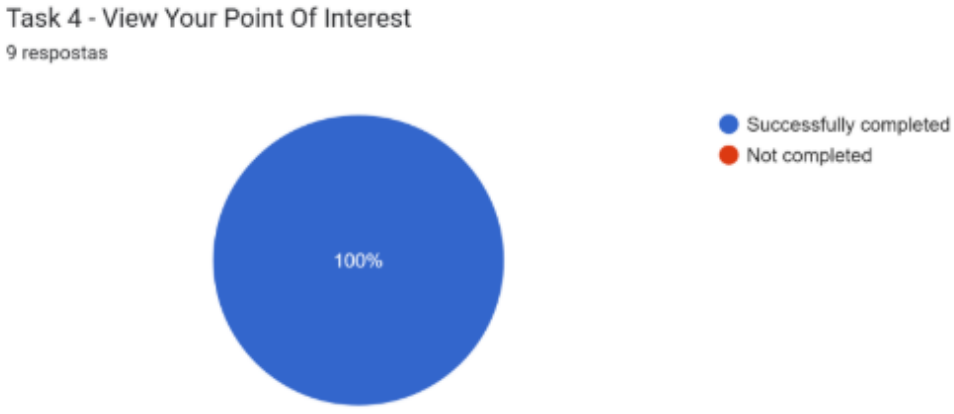


Figure 18: Task 4 - View Your Point Of Interest

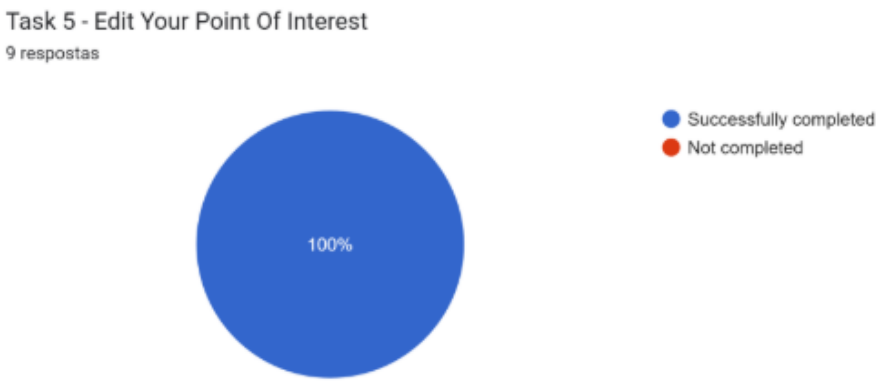


Figure 19: Task 5 - Edit Your Point Of Interest

10.1.2 System Usability Scale (SUS)

To measure the perceived usability of the ReliveThePast application, the System Usability Scale (SUS) was used. This standardized questionnaire consists of 10 statements, alternating between positive and negative phrasing, to which users respond on a 5-point Likert scale (from 1 = Strongly Disagree to 5 = Strongly Agree).

Scoring Method:

- For positively worded questions (odd-numbered: Q1, Q3, Q5, Q7, Q9), the score contribution is calculated as: Score = Response - 1
- For negatively worded questions (even-numbered: Q2, Q4, Q6, Q8, Q10), the score contribution is calculated as: Score = 5 - Response

Each individual's adjusted scores from all 10 questions are then summed, giving a value between 0 and 40. To convert this into a standard SUS score (ranging from 0 to 100), the total is multiplied by 2.5.

In this evaluation, responses were collected from 9 participants.

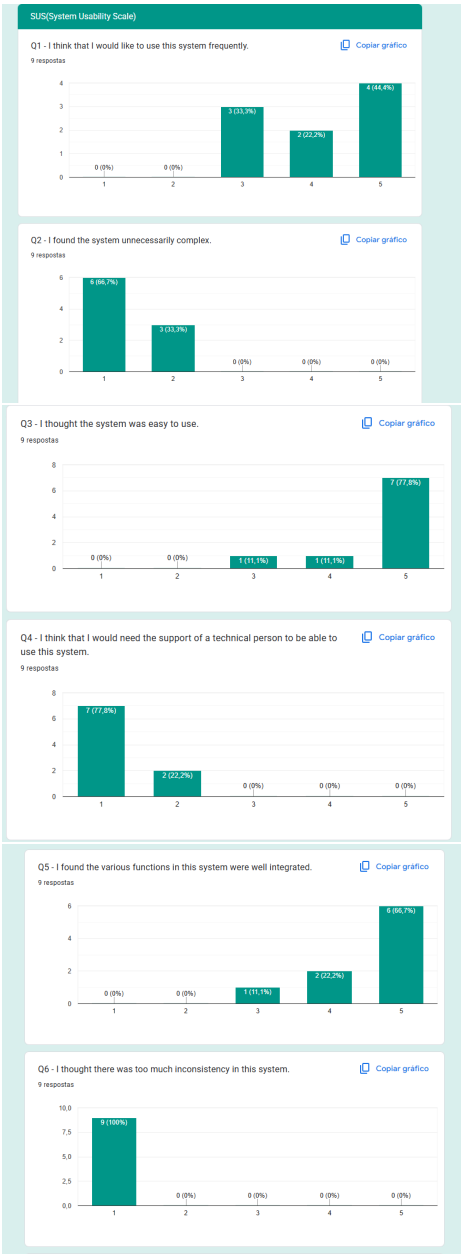


Figure 20: Q1, Q2, Q3, Q4, Q5, Q6

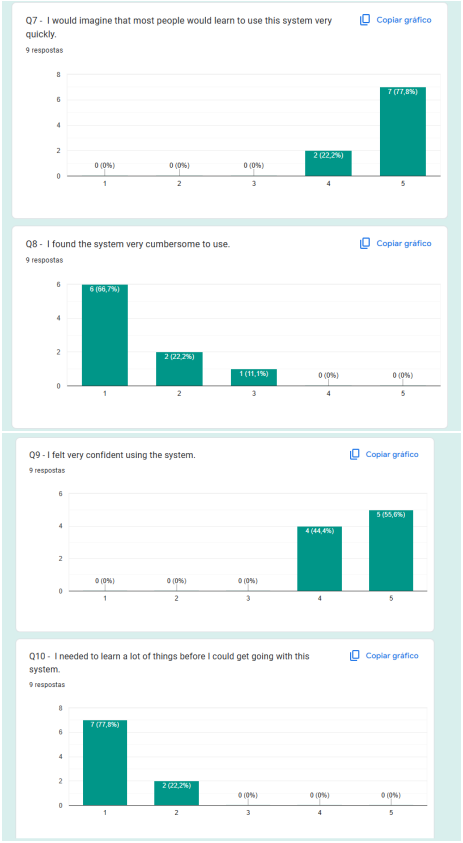


Figure 21: Q7, Q8, Q9, Q10

The average SUS score across all users was calculated, resulting in a final score of: **SUS Score = 86.1**

With a final SUS score of 86.1, the ReliveThePast application demonstrates an excellent level of usability. This places it well above the industry benchmark of 68, indicating that users found the app to be not only easy to use but also efficient, intuitive, and satisfying. In usability terms, this score suggests a product that users are likely to adopt, recommend to others, and use consistently without the need for technical assistance. The result confirms that the design and interaction model of the app are well-aligned with user expectations, supporting both the functional goals and user experience objectives of the project.

10.1.3 NPS (Net Promoter Score)

A Net Promoter Score (NPS) survey was created in order to measure the users' loyalty and to find out how satisfied they were of the ReliveThePast application. Subjects were asked how likely they were to recommend the app to a friend or colleague using a 0 to 10 scale. Participants were categorized into promoters (scoring 9–10), neutrals (scoring 7–8) and detractors (scoring 0–6). The resulting NPS is determined by subtracting the percentage of Detractors from the percentage of Promoters.

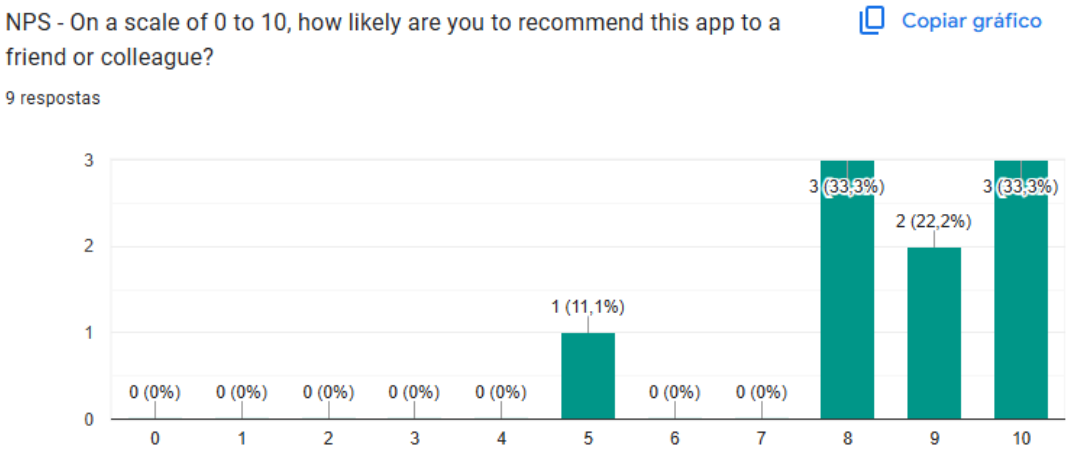


Figure 22: NPS (Net Promoter Score)

This survey resulted in NPS score of 44.4, which means users liked it. Such a score indicates that most of the users not only had a pleasure experience and are likely to be active advocates of the application to other potential users. While it's not within "excellent" territory (50 and above), it's already vastly ahead of the normal benchmark (0) and shows that people trust and like ReliveThePast. These results confirm the value of the app for design and use for the user and demonstrate possible generalization.

10.1.4 Customer Satisfaction Score (CSAT)

To assess the customer satisfaction of ReliveThePast overall feedback from users and a Customer Satisfaction Score (CSAT) survey was taken. The app had a single prompt in order: "How satisfied are you with the app?", and answers were given on a 5-point scale, ranging from 1 = "very dissatisfied" to 5 = "very satisfied." CSAT is a commonly used metric that's usually measured directly after a user has interacted with your product.

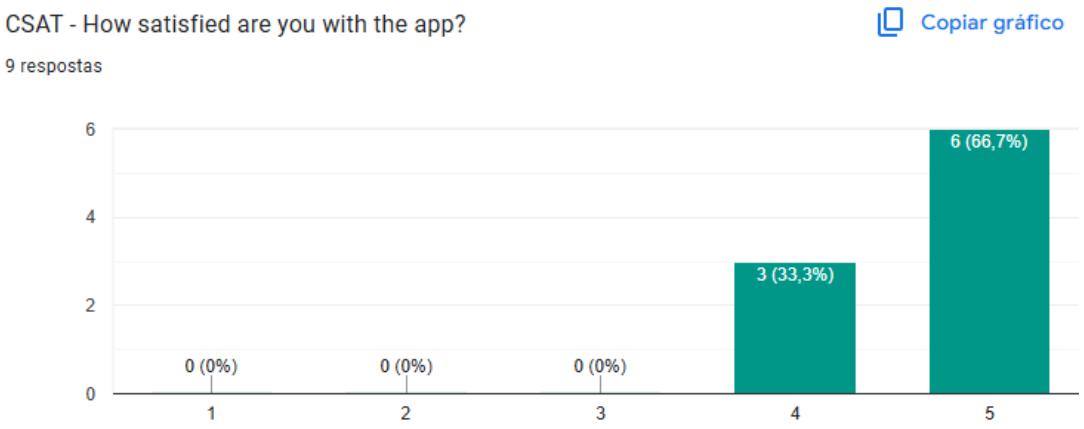


Figure 23: Customer Satisfaction Score (CSAT)

The outcomes demonstrated CSAT score for all subjects was 100, indicating all participants chose a 4 or 5 in the satisfaction scale. This flawless score suggests a great deal of satisfaction around the app's usability, functions, and user experience as a whole, by all employee types in non-production industries. Although CSAT indicates immediate satisfaction and not loyalty over time, this implies that the app not only meets but also exceeds the user's goals or expectations, sending only positive signals. It is a trend which accentuates the results from SUS and NPS scores as well, and supports that ReliveThePast is well suited to the needs of users.

10.1.5 Grading

In addition to formal usability metrics, users were also asked to provide a general rating for the ReliveThePast application. The app received an impressive average rating of 4.67 out of 5, with 66.7 of users giving it a perfect score of 5 and the remaining 33.3 rating it a 4. No ratings fell below 4, indicating a consistently positive perception across all respondents. This strong user approval further reinforces the app's usability, design quality, and value, complementing the favorable results from the SUS, NPS, and CSAT evaluations.

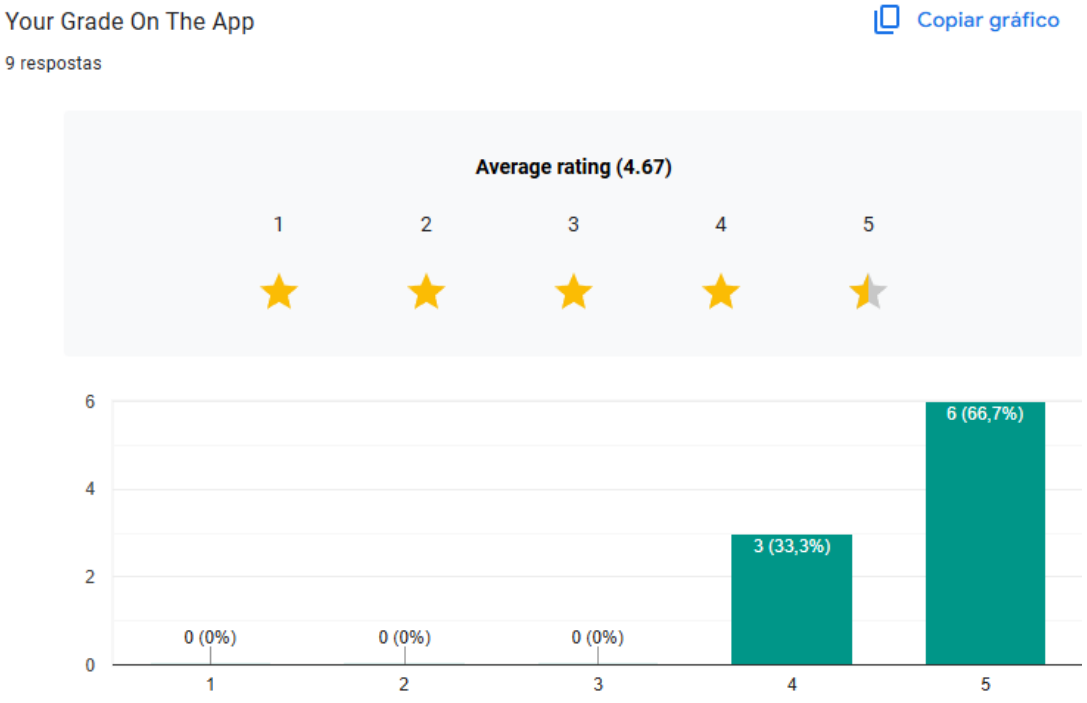


Figure 24: Grading the app

10.2 Compliance with requirements

To assess how much the use cases were implemented in the application with respect to its requirements, the defined use cases were compared with the functionalities of ReliveThePast. Table 43 provides an overview of this evaluation and shows whether each core requirement was addressed. Nearly all design objectives were achieved in full, with a high concordance to the original project aims. The only two features that didn't make it into the final implementation were notifications and reporting – both of which were de-scoped in favor of shipping on time and because they weren't critical for the initial release.

Requirement	**Implemented**
Search POIs	Yes
View POI Details	Yes
Save POIs	Yes
Rate and Review POIs	Yes
Receive Notifications	No
Manage User Accounts	Yes
Admin POI Management	Yes
Moderate Reviews	Yes
Generate Reports	No
Geolocation Services	Yes

Table 43: Compliance with functional requirements

Conclusion

11.1 Conclusion

The objective of this project was to develop a mobile application aimed at enriching cultural tourism and education by augmenting the experience of visiting historical and cultural Points of Interest (POIs). The primary objective was to create an intuitive and playful platform—ReliveThePast—using geolocation and multimedia in order to give users a captivating, educational and meaningful experience of the surroundings. And this very from day 1 it's been about usable, accessible, communicating of meaning.

All phases of development (from requirements elicitation and system architecture, to implementation and testing) were covered in the report. The feature design was developed based on user requirements and tested by the practice in a variety of methods including usability test, SUS, NPS, CSAT feedback etc. We obtained strong user satisfaction and very good usability scores in our evaluation, suggesting that the application serves its purpose well. Most of the features planned were indeed developed, excluding only some specific services such as notifications or reporting tools, which were considered unnecessary in first version.

In summary, ReliveThePast is a great example of how mobile technology can facilitate cultural discovery and learning in contemporary society. The results of the work not only meet the initial goals but will ensure a solid basis for further developments in the future. Based on its user-oriented design and its positive evaluation outcome, the system is well-placed for further development and take-up in the domains of tourism, education and heritage promotion.

11.2 Future Work

While ReliveThePast met some of the core goals it set out to accomplish, there were also a number of features we never managed to get working for the final release. Among these features has introduced the ability to share POIs, receive notifications in real time, and build a report system. While these are not required for the MVP, they are possible areas for future development to make users more engaged, personal and smart.

Future potential There are several possibilities to extend the purpose and effect of the application. Possible enhancements are better filtering and search functionality to narrow down the POI search by offering advanced filters, real-time crowd density information to avoid crowded places, etc. Creating educational and institutional partnerships might enhance the resources in the app with expert validated material and the potential use in schools, universities, and even tourism programs. Another promising approach is the fusing the digital and physical experience by allowing tourists to find tours offline.

These instructions are intended to enhance the usefulness of the app and to make it more relevant and applicable for a variety of users and settings. The avenues to development moving forward should be user-centered, data-driven, and open to partnership opportunities that further the educational and culture impact of the platform.

Bibliography

[1] Teacher's Document (Modelo de Documento de Especificação de Requisitos) . Available through the course materials provided on Moodle.

[2] Teacher's Document (Modelo de Relatório de Engenharia de Software - Arquitectura de SW Ficheiro) . Available through the course materials provided on Moodle.

[3] Teacher's Document (Template geral (Projecto I e Projecto II)) . Available through the course materials provided on Moodle.

[4] Overleaf Documentation. "Getting Started with LaTeX". https://www.overleaf.com/learn/latex/.

[5] Graphviz - Graph Visualization Software. "Documentation and Examples". https://graphviz.org/.

[6] WebGraphviz - Online Graph Visualization Tool. http://www.webgraphviz.com.

[7] TeamGantt - Online project management tool that allows teams to create, track, and collaborate on Gantt charts for effective task planning and scheduling. https://app.teamgantt.com/

Acknowledgements

I want to take this opportunity to thank all the people who has helped me develop this project.

I want to express my gratitude towards my family and friends for their endless affirmation, tolerance and emotional support throughout all phases of this work. What made me to continue fighting was because most of them believe in me so I just need to make their beliefworth.

I also extend my heartfelt appreciation to my teachers and academic mentors, whose guidance, feedback, and knowledge were fundamental to the success of this project. Their dedication to teaching and their support throughout this academic journey were instrumental in helping me grow not only as a developer, but also as a problem solver.